



به نام خدا
دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس شبکه‌های عصبی و یادگیری عمیق

تمرین دوم

پرسش ۱	نام و نام خانوادگی	بابک حسینی محتشم
	شماره دانشجویی	۸۱۰۱۰۱۴۰۸
پرسش ۲	نام و نام خانوادگی	
	شماره دانشجویی	
	مهلت ارسال پاسخ	۱۴۰۴.۰۱.۲۶

فهرست

پرسش ۱. تشخیص بیماران مبتلا به COVID-۱۹ با استفاده از تصاویر X-Ray.....	۱
۱-۱. تحلیل دیتاست.....	۱
۲-۱. بررسی تعادل کلاس‌ها.....	۱
۳-۱. پیش‌پردازش داده‌ها.....	۳
۴-۱. آماده‌سازی مدل.....	۵
۵-۱. آموزش و ارزیابی مدل.....	۷
۱-۵-۱. آموزش پیش از اضافه کردن داده‌های افزونه.....	۹
۲-۵-۱. آموزش با نرخ یادگیری ثابت.....	۱۰
۳-۵-۱. آموزش با نرخ یادگیری متغیر.....	۱۴
۴-۵-۱. ایجاد داده‌های افزونه با روش مقاله.....	۲۱
۵-۵-۱. ارزیابی مدل‌های دیگر برای افزایش عملکرد.....	۲۳
۶-۵-۱. ارزیابی نهایی.....	۲۳
۶-۱. یادگیری انتقالی.....	۲۵
۱-۶-۱. تصاویر ImageNet.....	۲۶
۲-۶-۱. VGG16.....	۲۶
۳-۶-۱. MobileNetV2.....	۲۹
۷-۱. جمع‌بندی.....	۳۱

شکل‌ها

- شکل ۱. نمونه‌ای از دادگان ۱
- شکل ۲. توزیع کلاس‌های دادگان آموزش و تست ۱
- شکل ۳. نمونه‌ای از data augmentaion ۴
- شکل ۴. شکل شماره ۴ مقاله نشان‌دهنده معماری مدل CNN ۵
- شکل ۵. خطای مدل روی داده متعادل شده ۹
- شکل ۶. صحت مدل روی داده متعادل شده ۹
- شکل ۷. ماتریس درهم‌ریختگی مدل روی داده متعادل شده ۹
- شکل ۸. خطای داده افزونه شده با نرخ یادگیری ۰.۰۰۰۱ ۱۰
- شکل ۹. صحت داده افزونه شده با نرخ یادگیری ۰.۰۰۰۱ ۱۰
- شکل ۱۰. ماتریس درهم‌ریختگی روی داده افزونه شده با نرخ یادگیری ۰.۰۰۰۱ ۱۱
- شکل ۱۱. خطای داده افزونه شده با نرخ یادگیری ۰.۰۰۱ ۱۲
- شکل ۱۲. صحت داده افزونه شده با نرخ یادگیری ۰.۰۰۱ ۱۲
- شکل ۱۳. ماتریس درهم‌ریختگی روی داده افزونه شده با نرخ یادگیری ۰.۰۰۱ ۱۲
- شکل ۱۴. خطای داده افزونه شده با نرخ یادگیری ۰.۰۱ ۱۳
- شکل ۱۵. صحت داده افزونه شده با نرخ یادگیری ۰.۰۱ ۱۳
- شکل ۱۶. ماتریس درهم‌ریختگی روی داده افزونه شده با نرخ یادگیری ۰.۰۱ ۱۳
- شکل ۱۷. تغییرات نرخ یادگیری طبق CosineDecay ۱۵
- شکل ۱۸. خطای مدل با نرخ یادگیری CosineDecay ۱۶
- شکل ۱۹. صحت مدل با نرخ یادگیری CosineDecay ۱۶
- شکل ۲۰. ماتریس درهم‌ریختگی مدل با نرخ یادگیری CosineDecay ۱۶
- شکل ۲۱. تغییرات نرخ یادگیری طبق ExponentialDecay ۱۷
- شکل ۲۲. خطا با نرخ یادگیری ExponentialDecay ۱۷
- شکل ۲۳. صحت با نرخ یادگیری ExponentialDecay ۱۷
- شکل ۲۴. ماتریس درهم‌ریختگی مدل با نرخ یادگیری ExponentialDecay ۱۸
- شکل ۲۵. تغییرات نرخ یادگیری طبق CosineDecayRestarts ۱۹
- شکل ۲۶. خطا با نرخ یادگیری CosineDecayRestart ۱۹

- شکل ۲۷. صحت با نرخ یادگیری CosineDecayRestart ۱۹
- شکل ۲۸. ماتریس درهم ریختگی مدل با نرخ یادگیری CosineDecayRestarts ۲۰
- شکل ۲۹. نمونه‌ای از data augmentation مشابه مقاله ۲۱
- شکل ۳۰. تغییرات خطا با افزودن روش مقاله ۲۲
- شکل ۳۱. تغییرات صحت با افزودن روش مقاله ۲۲
- شکل ۳۲. ماتریس درهم ریختگی با افزودن روش مقاله ۲۲
- شکل ۳۳. تغییرات خطا روی داده تست ۲۴
- شکل ۳۴. تغییرات صحت ارزیابی روی داده تست ۲۴
- شکل ۳۵. ماتریس درهم ریختگی برای ارزیابی روی داده تست ۲۴
- شکل ۳۶. تغییرات خطای VGG16 روی داده تست ۲۸
- شکل ۳۷. تغییرات صحت VGG16 روی داده تست ۲۸
- شکل ۳۸. ماتریس درهم ریختگی VGG16 برای ارزیابی روی داده تست ۲۸
- شکل ۳۹. تغییرات صحت MobileNetV2 ۳۰
- شکل ۴۰. تغییرات خطای MobileNetV2 ۳۰
- شکل ۴۱. ماتریس درهم ریختگی MobileNetV2 برای ارزیابی روی داده تست ۳۰

جدول‌ها

- جدول ۱. جدول شماره ۳ مقاله نشان‌دهنده پارامترهای مدل ۵
- جدول ۲. خلاصه معماری مدل بدون لایه‌های Pooling,Dropout,BN ۶
- جدول ۳. معیارهای ارزیابی مدل روی داده متعادل شده ۱۰
- جدول ۴. معیارهای ارزیابی مدل روی داده افزونه شده با نرخ یادگیری ۰.۰۰۰۱ ۱۱
- جدول ۵. معیارهای ارزیابی مدل روی داده افزونه شده با نرخ یادگیری ۰.۰۰۱ ۱۳
- جدول ۶. معیارهای ارزیابی مدل روی داده افزونه شده با نرخ یادگیری ۰.۰۱ ۱۴
- جدول ۷. معیارهای ارزیابی مدل با نرخ یادگیری CosineDecay ۱۶
- جدول ۸. معیارهای ارزیابی مدل با نرخ یادگیری ExponentialDecay ۱۸
- جدول ۹. معیارهای ارزیابی مدل با نرخ یادگیری CosineDecayRestarts ۲۰
- جدول ۱۰. معیارهای ارزیابی مدل با افزودن به روش مقاله ۲۲
- جدول ۱۱. معیارهای ارزیابی مدل برای ارزیابی روی داده تست ۲۴
- جدول ۱۲. خلاصه معماری مدل تشکیل شده از VGG16 برای یادگیری انتقالی ۲۷
- جدول ۱۳. معیارهای ارزیابی مدل VGG16 برای ارزیابی روی داده تست ۲۸
- جدول ۱۴. خلاصه معماری مدل تشکیل شده از MobileNetV2 برای یادگیری انتقالی ۲۹
- جدول ۱۵. معیارهای ارزیابی مدل MobileNetV2 برای ارزیابی روی داده تست ۳۱
- جدول ۱۶. مقایسه سه مدل با ارزیابی روی داده تست ۳۲

پرسش ۱. تشخیص بیماران مبتلا به COVID-۱۹ با استفاده از تصاویر X-Ray

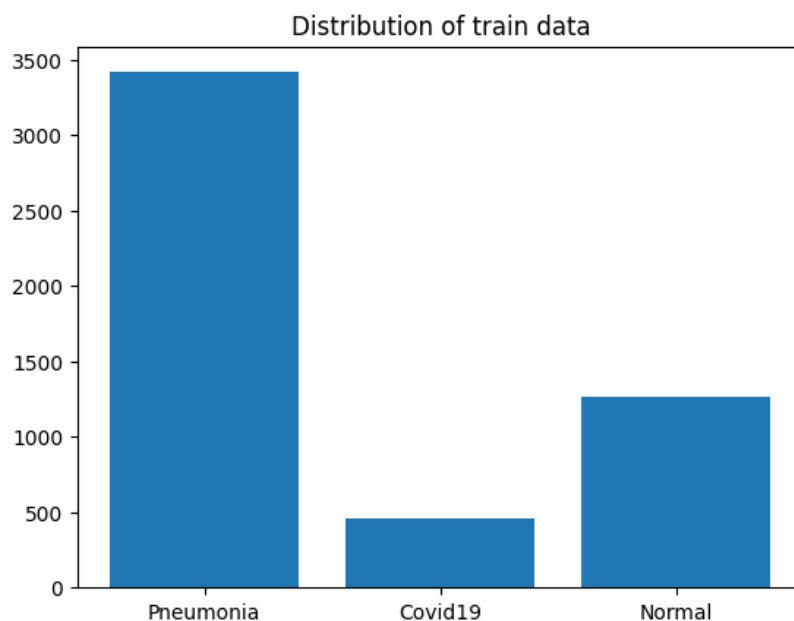
۱-۱. تحلیل دیتاست

دادگان شامل سه کلاس COVID19، PNEUMONIA و NORMAL است که به ترتیب نشان‌دهنده تصاویر X-Ray از ریه بیماران مبتلا به کرونا، ذات‌الریه و افراد سالم است. تمامی تصاویر به فرمت jpg هستند. دادگان آموزش شامل ۳۴۱۸ نمونه از کلاس ذات‌الریه، ۴۶۰ نمونه از کلاس کووید ۱۹ و ۱۲۶۶ نمونه از افراد سالم است. دادگان تست هم توزیع مشابهی دارند و تعداد نمونه‌های کلاس‌های آن به ترتیب ۸۵۵، ۱۱۶ و ۳۱۷ است. همچنین ابعاد تصاویر بسیار متغیر است و حداقل ۱۲۷ و حداکثر ۵۶۲۳ پیکسل می‌تواند باشد.



شکل ۱. نمونه‌ای از دادگان

۱-۲. بررسی تعادل کلاس‌ها



شکل ۲. توزیع کلاس‌های دادگان آموزش و تست

همان طور که مشخص است توزیع کلاس‌ها متعادل نیست و این می‌تواند فرایند یادگیری را سخت‌تر کند زیرا مدل با دیدن دادگان یاد می‌گیرد و اختلاف در تعداد دادگان دو کلاس می‌تواند باعث اختلاف در خطای کلاس‌ها شود و باعث بیش‌برازش یا کم‌برازش برای کلاس‌ها شود. همچنین ممکن است مدل به دلیل برابر نبودن تعداد نمونه‌ها درصد بالایی بگیرد ولی در تشخیص کلاس‌های با نمونه کمتر دقت نامقبولی داشته باشد.

یکی از کتابخانه‌هایی که امکانات زیادی برای رفع مشکل متعادل نبودن کلاس‌ها به ما می‌دهد imbalanced-learn نام دارد. به طور کلی برخی مدل‌ها نسبت به برخی دیگر نسبت به متعادل بودن کلاس‌ها حساسیت متفاوتی دارند و به همین دلیل می‌توان با تغییر مدل عملکرد را افزایش داد. همچنین برخی مدل‌ها امکان افزایش وزن برای برخی داده‌ها را دارند تا اهمیت بیشتری به آن داده‌ها بدهند که می‌تواند در چنین شرایطی باعث بهبود عملکرد شود.

همچنین استفاده از متریک مناسب برای مقایسه و خطای مناسب برای آموزش مدل‌ها در مواجهه با دادگان نامتعادل ضروری است. استفاده از precision و recall که دو نوع خطای متفاوت مدل را مبنای عملکرد می‌دهند می‌تواند بر اساس هدف و اهمیت کلاس‌ها باعث بهبود عملکرد شوند. متریک‌هایی مثل F1-score و ROC و AUC هم با در نظر گرفتن خطاهای مختلف، در صورت کم بودن خطا در یک کلاس ولی زیاد بودن خطا برای کلاسی دیگر، عملکردی پایین به مدل می‌توانند نسبت دهند.

به طور کلی دو دسته روش رایج هم وجود دارد که با تغییر در دادگان همراه است. قبل از توضیح ذکر این نکته ضروری است که مانند اکثر فرایندهای دیگر در یادگیری، نباید اطلاعاتی از دادگان تست به دست بیاوریم و بدین ترتیب این روش‌ها را تنها روی دادگان آموزش پیاده می‌کنیم و یا روی دادگان آموزش و تست به طور جداگانه پیاده می‌کنیم.

روش اول Undersampling است که همان طور که از نامش معلوم است در این روش از کلاس‌هایی که دادگان زیادی دارند، تعدادی از نمونه‌ها را حذف می‌کنیم تا تعداد دادگان تمام کلاس‌ها نزدیک به هم شود. اصولاً این روش در اکثر مواقع توصیه نمی‌شود زیرا حذف دادگان می‌تواند باعث حذف اطلاعات مهم برای یادگیری و در نتیجه افت عملکرد مدل شود ولی برای دادگانی که نمونه‌ها بسیار به هم شبیه هستند و تعدادشان بسیار زیاد است و فکر می‌کنیم با حذف تعدادی از آن‌ها اطلاعاتی مفیدی از دست نمی‌دهیم می‌توانیم از این روش استفاده کنیم.

یکی از الگوریتم‌های روش بالا Random undersampling است که به طور تصادفی از کلاس با نمونه بالا داده انتخاب می‌کنیم و این داده‌ها را حذف می‌کنیم. الگوریتم بهتر Tomek Links است که نمونه‌های دو

کلاس مختلف که بسیار ویژگی‌های مشابهی دارند را پیدا می‌کند و نمونه از کلاس پرجمعیت‌تر را حذف می‌کند.

دسته‌ای دیگر از روش‌ها به نام Upsampling وجود دارند که برخلاف روش‌های undersampling، که داده از کلاس پرجمعیت حذف می‌کردند، این روش‌ها داده‌هایی از کلاس کم جمعیت تولید می‌کنند و به همین دلیل اطلاعاتی از بین نمی‌رود و می‌توانند عملکرد بهتری داشته باشند. یکی از معایب این روش‌ها این است که با افزایش نمونه‌های ساختگی یک کلاس ممکن است باعث رخداد بیش‌برازش شویم و راه‌حل آن این است که تفاوت‌هایی بین داده‌گانی که تولید می‌کنیم و داده‌گان موجود ایجاد کنیم.

الگوریتم Random upsampling به طور تصادفی نمونه‌هایی از کلاس با نمونه کم‌تر انتخاب می‌کند و آن نمونه را کپی می‌کند. این کار را می‌توان با جایگزینی یا بدون جایگزینی نمونه‌های انتخاب شده انجام داد. الگوریتم بهتر Smote نام دارد که ابتدا تعدادی از نمونه‌های کلاس کم جمعیت‌تر را انتخاب می‌کند و به ازای هر نمونه K همسایه نزدیک به آن که در همان کلاس هستند را در نظر می‌گیریم و به ازای هر داده همسایه، با درون‌یابی نمونه‌هایی بین این دو نمونه تولید می‌کنیم.

البته ترکیب روش‌های بالا می‌تواند با پوشش دادن نقطه ضعف هر روش، باعث بهبود کلی شود. یکی از روش‌های ترکیبی SMOTE-Tomek است که ابتدا با روش Smote تعداد نمونه‌ها را زیاد می‌کند و سپس با روش Tomek برخی نمونه‌ها را حذف می‌کند.

۱-۳. پیش‌پردازش داده‌ها

بعضی اوقات داده‌های موجود برای آموزش مدل بسیار کم هستند و در این‌جور مواقع استفاده از روش Data augmentation می‌تواند با افزایش تعداد داده‌ها، باعث امکان آموزش مدل با عملکرد مطلوب شود. به طور کلی با کمک روش‌های Data augmentation، تغییراتی در داده‌ها ایجاد می‌کنیم که این تغییرات در حدی هستند که باز هم به داده ایجاد شده پس از تغییرات را بتوانیم برچسب یکسانی بدهیم پس با این تغییرات ویژگی‌های اصلی داده‌ها را حفظ می‌کنیم و ویژگی‌هایی که ممکن است در داده‌های واقعی متفاوت باشند را تغییر می‌دهیم.

از مزیت‌های اصلی این روش افزایش قدرت تعمیم مدل و کاهش بیش‌برازش است و مدل نسبت به تغییراتی که ایجاد کردیم مقاوم می‌شود. همچنین به کمک این روش‌ها نیاز به جمع‌آوری داده‌های فراوان چند برابر کاهش می‌یابد و بدین ترتیب می‌توان مدل‌های پیچیده‌تر را با داده‌های کمتر نیز آموزش داد.

یکی از کاربردهای اصلی این روش برای داده‌هایی است که جمع‌آوری آن‌ها سخت است یا تعدادی کمی از آن موجود است برای مثال برای پیش‌بینی بیماری‌های نادر می‌توان از این روش استفاده کرد. به طور کلی

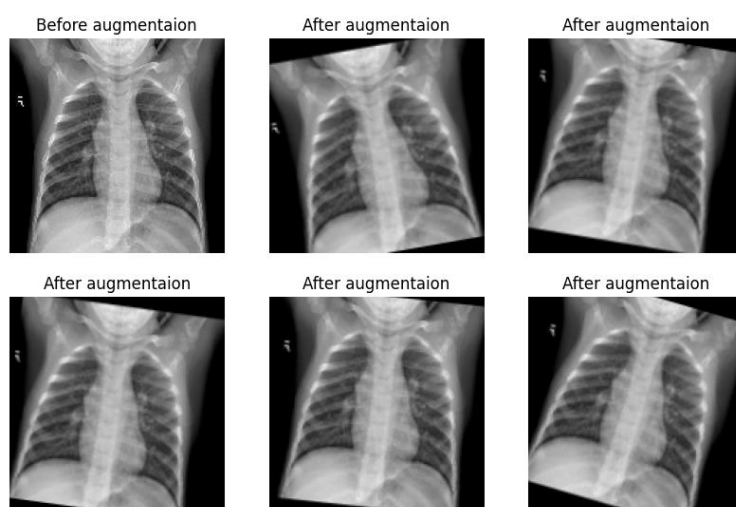
این روش در مواقعی که با داده‌های تصویری سر و کار داریم در حوزه بینایی ماشین رایج است چرا که نسبت داده‌های تصاویر به ویژگی‌های آن‌ها که ابعاد است بسیار کم است و می‌توان با تغییراتی در نور یا تغییرات هندسی تصویر، تصاویری با برچسب یکسان تولید کرد و بدین ترتیب مقاوت مدل را نسبت به تغییرات بین تصاویر گرفته شده در شرایط مختلف افزایش داد.

یکی از تغییرات رایج بازتاب افقی یا عمودی تصویر است. برای کاربردهای پزشکی مشابه این دادگان معمولاً تنها بازتاب نسبت به محور عمودی داده می‌شود چرا که بدن انسان نسبت به محور عمودی تا حدودی متقارن است. در مقاله مرجع نیز از این نوع بازتاب استفاده شده است ولی از نظر ما چون در تمامی دادگان تصویربرداری بدین صورت، تصویر برداری از جلو انجام می‌شود به دلیل همین نیازی به افزایش مقاوت نسبت به بازتاب نیست چرا که همچنین تصاویری در دادگان وجود نخواهد داشت.

تغییر در زاویه تصاویر نیز رایج است و در مقاله داده شده، تغییر زاویه به صورت مضارب ۹۰ درجه صورت گرفته است. به دلایل مشابه بازتاب، تغییر درجه بالا نزدیکی ۹۰ درجه در داده‌های واقعی رخ نمی‌دهد پس ما هم تغییر درجه را به میزان اندک اعمال می‌کنیم.

یکی دیگر از تبدیلاتی که ما استفاده می‌کنیم، جابه‌جایی است زیرا ممکن است ریه در مرکز تصویر نباشد ولی این تغییر در مقاله اصلی اعمال نشده است. در بخش‌های دیگر مدل‌هایی را با دادگانی که با روش‌های مشابه مقاله augment شده‌اند، آموزش خواهیم داد و بررسی خواهیم کرد.

به جز تبدیلات هندسی، تغییر نور و کنتراست نیز می‌توان انجام داد که به نظر ما ممکن است باعث افزایش خطای مدل شود چرا که تصاویر X-ray قاعدتاً استانداردهای خاصی دارند و تغییر در نور و کنتراست ممکن است باعث تغییر تشخیص شود.



شکل ۳. نمونه‌ای از data augmentaion

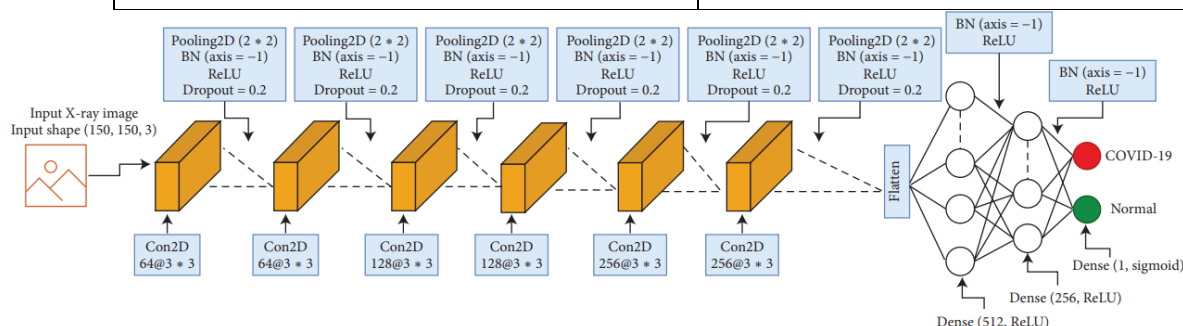
در مقاله داده شده، تعداد تصاویر ۵ برابر شده است ما نیز تعداد تصاویر را ۵ برابر می‌کنیم زیرا با انتخاب تعدادی فرد، انتظار داریم که بدون در نظر گرفتن خود تصویر اصلی، تعداد برابر از زوایای مثبت و منفی و تغییرات مکانی در هر جهتی داشته باشیم. باید توجه کرد که داده‌های تست و validation را augment نمی‌کنیم و آن‌ها را پیش از augment کردن داده آموزش جدا کردیم تا هیچ اطلاعاتی نسبت به آن‌ها به دست نیاوریم. بدین ترتیب ۸۹۷ داده آموزش، را تبدیل به ۴۴۸۵ داده آموزش کردیم. همچنین ۴۸۳ داده validation و ۳۴۸ داده تست هم داریم.

۴-۱. آماده‌سازی مدل

در مقاله جزئیات مدل بدین صورت به تصویر کشیده شده است.

جدول ۱. جدول شماره ۳ مقاله نشان‌دهنده پارامترهای مدل

Parameter	Value
Input dimension	(150, 150, 3)
Filter to learn	64, 128, 256
Max pooling	2×2
Batch normalization	Axis = -1
Activation functions	ReLU, sigmoid
Dropout rate	20%
Kernel size	3×3
Epochs	50
Optimizer	Adam
Loss function	binary_crossentropy



شکل ۴. شکل شماره ۴ مقاله نشان‌دهنده معماری مدل CNN

با توجه به تفاوت در دادگان، نیاز به ایجاد تفاوت‌هایی در مدل داشتیم. با توجه به وجود دو برچسب بیمار کرونا یا فرد سالم در دادگان مقاله خروجی مدل مقاله یک نورون دارد ولی در دادگان ما سه برچسب وجود دارد و به همین دلیل در لایه آخر سه نورون قرار می‌دهیم. به دلیل مشابه به جای استفاده از تابع فعال‌سازی sigmoid در لایه آخر، ما از تابع فعال‌سازی softmax استفاده می‌کنیم. همچنین تابع خطای استفاده شده نیز برای دادگان شامل دو کلاس است ولی برای دادگان سه کلاسه‌ای که ما استفاده می‌کنیم، کار نخواهد کرد پس از sparse_categorical_crossentropy استفاده می‌کنیم.

پس از ایجاد مدل متوجه شدیم، که پس از گذر داده از لایه آخر، ابعاد داده صفر می‌شود و بدین ترتیب ارور دریافت می‌کنیم. در مقاله توضیحی در این مورد داده نشده است و این مشکلی نیست که به دلیل تفاوت در دادگان پیش آمده باشد چرا که ابعاد تصاویر را مشابه مقاله کرده‌ایم. پس ما برای رفع این مشکل، برای تمام لایه‌های کانولوشنی padding="same" قرار دادیم تا سطرها و ستون‌هایی با مقادیر صفر اطراف تصاویر اضافه شوند و بدین ترتیب از صفر شدن ابعاد تا لایه آخر جلوگیری می‌کنیم.

به دلیل تعداد لایه‌های زیاد، در تصویر زیر تنها بخشی از معماری مدل را مشاهده می‌کنیم.

جدول ۲. خلاصه معماری مدل بدون لایه‌های Pooling, Dropout, BN

Layer (type)	Output Shape	Param #
conv2d_204 (Conv2D)	(None, 150, 150, 64)	1,792
conv2d_205 (Conv2D)	(None, 75, 75, 64)	36,928
conv2d_206 (Conv2D)	(None, 37, 37, 128)	73,856
conv2d_207 (Conv2D)	(None, 18, 18, 128)	147,584
conv2d_208 (Conv2D)	(None, 9, 9, 256)	295,168
conv2d_209 (Conv2D)	(None, 4, 4, 256)	590,080
flatten_34 (Flatten)	(None, 1024)	0
dense_102 (Dense)	(None, 512)	524,800
dense_103 (Dense)	(None, 256)	131,328
dense_104 (Dense)	(None, 3)	771

Total params: 1,808,963 (6.90 MB)

Trainable params: 1,805,635 (6.89 MB)

Non-trainable params: 3,328 (13.00 KB)

۵-۱. آموزش و ارزیابی مدل

ما برای جلوگیری از بیش‌برازش و کم‌برازش مدل، از early stopping استفاده می‌کنیم بدین صورت که مدل را ۴۰ اپیاک آموزش می‌دهیم و سپس تا هنگامی که ۲۵ اپیاک متوالی خطای مدل روی داده validation کاهش نیابد، آموزش را ادامه می‌دهیم. دلیل انتخاب این اعداد هم این است که طبق تست‌هایی که انجام دادیم، مدل‌هایی که بالای ۸۰ اپیاک آموزش می‌دیدند تا ۵ درصد عملکرد بهتری نسبت به مدل‌هایی که کمتر از ۵۰ اپیاک آموزش می‌دیدند داشتند. البته اگر خطا مدام کاهش پیدا کند نیز پس از ۱۰۰ اپیاک آموزش متوقف می‌شود. همچنین batch size را در آموزش این مدل‌ها مقدار رایج ۳۲ در نظر گرفتیم.

برای ارزیابی مدل‌ها از چهار معیار معروف استفاده خواهیم کرد. معمولاً معیارها برای تشخیص بین دو کلاس تعریف می‌شوند ولی می‌توان برای تشخیص بیش از دو مدل نیز از آن‌ها استفاده کرد. هر یک از این معیارها با کمک چهار معیار ساده‌تر توصیف می‌شوند: TP, FN, TN, FP. دو معیار FN و FP نشان‌دهنده تشخیص اشتباه و معیارهای TP و TN نشان‌دهنده تشخیص درست هستند. هنگامی که مدلی داده‌ای را به عنوان کلاس مورد نظر ما تشخیص می‌دهد، بر اساس درست بودن یا نبودن تشخیص از آن تشخیص را جزو FP یا TP در نظر می‌گیریم و اگر جزو کلاس دیگر تشخیص دهد، به طور مشابه تشخیص FN یا TN است. حال به بررسی معیارهای دیگر می‌پردازیم.

معیار اول صحت (accuracy) نام دارد. این معیار رایج‌ترین معیار در یادگیری ماشین است و نشان‌دهنده عملکرد کلی مدل است. این معیار از رابطه زیر به دست می‌آید:

$$accuracy = \frac{TP + TN}{FP + FN + TP + TN}$$

از رابطه بالا متوجه می‌شویم که از دید صحت ارزش تمام کلاس‌ها یکسان است و برای افزایش صحت یک مدل کافی است میزان تشخیص‌های درست را افزایش دهیم. مشکل اصلی این معیار در مواردی است که اهمیت کلاس‌ها یکسان نباشد که یکی از رایج‌ترین آن زمانی اتفاق می‌افتد که تعداد داده‌های کلاس‌ها نامتعادل باشد. برای مثال اگر مدلی تمام افراد را سالم تشخیص دهد و تنها یک درصد از افراد بیمار باشند، صحت ۹۹ درصد می‌شود در حالیکه هیچ بیماری را نتوانسته‌ایم به درستی تشخیص دهیم و در اصل انتظار داریم معیار ارزش کمی به مدل بدهد در صورتی که چنین اتفاقی با این معیار نمی‌افتد.

برای رفع مشکل بالا سراغ معیارهای دیگر می‌رویم. دو معیار دقت (precision) و بازیابی (recall) معمولاً براساس اهمیت کلاس مثبت یا منفی می‌توانند باعث ارزیابی بهتر مدل شوند. این دو معیار از رابطه‌های زیر به دست می‌آیند:

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$

می‌تواند دید هر دو مدل با افزایش تشخیص کلاس مثبت افزایش می‌یابند ولی مقدار دقت با تشخیص اشتباه داده به عنوان کلاس مثبت و در بازیابی با تشخیص اشتباه داده به عنوان کلاس منفی کاهش می‌یابند. اگر مثال بیماری را در نظر بگیریم، زیاد بودن تعداد TN باعث افزایش این دو معیار نمی‌شود. با این حال در این مثال استفاده از بازیابی توصیه می‌شود چرا که از بین دو خطای ممکن، کاهش FN اهمیت بسیار بیشتری دارد زیرا این خطا نشان‌دهنده این است که بعضی بیماران را به درستی تشخیص نداده‌ایم در صورتی که خطای دیگر نشان‌دهنده آن است که بعضی افراد سالم را به عنوان بیمار تشخیص داده‌ایم که ضرر آن کم‌تر است. به عنوان مثالی از جایی که استفاده از دقت توصیه می‌شود، می‌توان تشخیص ایمیل‌های اسپم را در نظر گرفت در این حالت اهمیت خطای FP بیشتر است چرا که نشان می‌دهد به اشتباه ایمیل‌های درست را به عنوان اسپم تشخیص داده‌ایم.

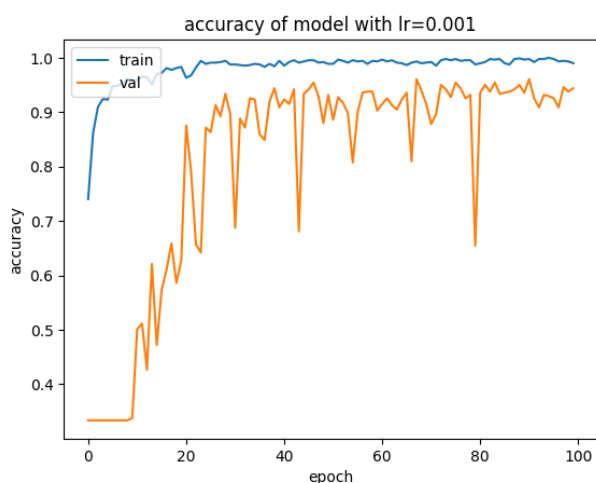
معمولاً افزایش هر دو معیار دقت و بازیابی به طور هم‌زمان به دلیل ماهیت متفاوت آن‌ها امکان پذیر نیست و در حالت‌هایی که هر دوی معیارها اهمیت بالایی دارند، می‌توانیم از معیار F1-score استفاده کنیم که برابر میانگین هارمونیک این دو معیار است و حتی اگر یکی از این دو معیار مقدار کمی داشته باشد، معیار F1-score نیز مقدار کمی خواهد داشت.

$$F1score = \frac{2 \times precision \times recall}{precision + recall}$$

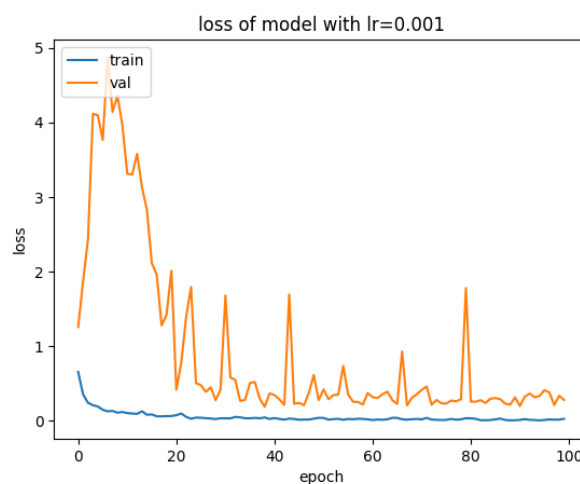
برای این پروژه به دلیل متعادل بودن کلاس‌ها استفاده از معیار صحت accuracy می‌تواند به ما در مقایسه مدل‌ها کمک کند البته که به دلیل تعداد برابر کلاس‌ها انتظار داریم بقیه معیارها هم مقادیر مشابهی داشته باشند. پس ما هم معیار صحت را به عنوان معیار اصلی برای این پروژه قرار می‌دهیم ولی بقیه معیارها را هم بررسی خواهیم کرد و در نظر داریم که در زمان کرونا، اهمیت معیار بازیابی کلاس کرونا اهمیت بیشتری داشته است.

۱-۵-۱. آموزش پیش از اضافه کردن داده‌های افزونه

در مقاله داده شده، پس از متعادل کردن کلاس‌ها و پیش از افزایش تعداد داده‌ها، دقت مدل به حدود ۷۰ درصد رسید. ما نیز برای بررسی دقت، پیش از augmentation مدل را آموزش دادیم. دو شکل زیر تغییرات خطا و صحت مدل را روی داده‌های آموزش و تست در ایپاک‌های مختلف حین آموزش نشان می‌دهند.

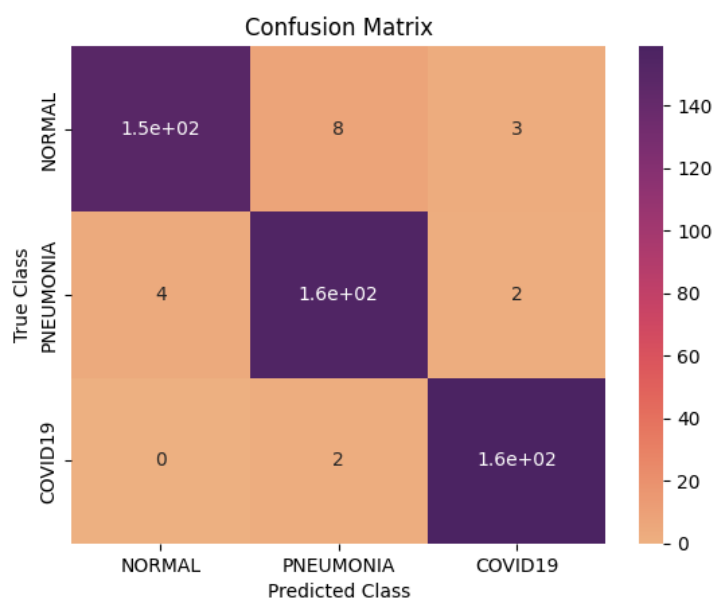


شکل ۶: صحت مدل روی داده متعادل شده



شکل ۵: خطای مدل روی داده متعادل شده

آموزش پس از ۱۰۰ ایپاک متوقف می‌شود. همان طور که انتظار داریم، با افزایش ایپاک‌ها خطای مدل روی هر دو داده کاهش و دقت افزایش پیدا کرده است.



شکل ۷: ماتریس درهم‌ریختگی مدل روی داده متعادل شده

جدول ۳. معیارهای ارزیابی مدل روی داده متعادل شده

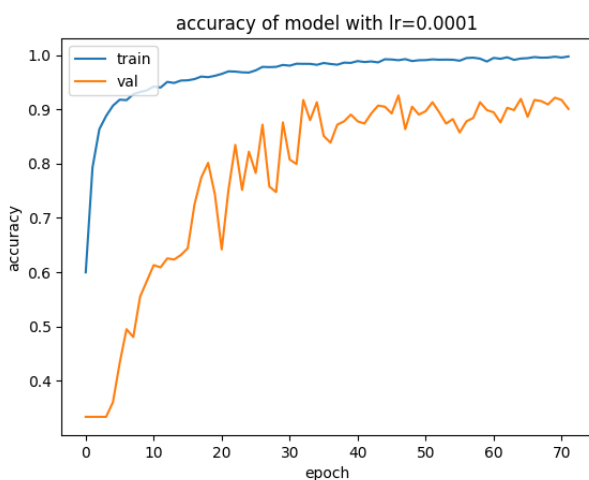
	NORMAL	PBEUMONIA	COVID19	Micro	Macro
Accuracy	۹۳.۱۷	۹۶.۲۷	۹۸.۷۶	۹۶.۰۷	۹۶.۰۷
Precision	۹۷.۴۰	۹۳.۹۴	۹۶.۹۵	۹۶.۰۷	۹۶.۱۰
Recall	۹۳.۱۷	۹۶.۲۷	۹۸.۷۶	۹۶.۰۷	۹۶.۰۷
F1-score	۹۵.۲۴	۹۵.۰۹	۹۷.۸۵	۹۶.۰۷	۹۶.۰۶

می‌توان دید که عملکرد مدل روی طبق تمام معیارها بالای ۹۳ درصد است که بسیار فراتر از ۷۰ درصدی است که مدل مقاله کسب کرده بود. همچنین می‌توان دید که اکثر اشتباهات در تشخیص بیماران دارای ذات‌الریه رخ داده است در صورتی که کلاس ذات‌الریه در داده‌های مقاله وجود نداشت.

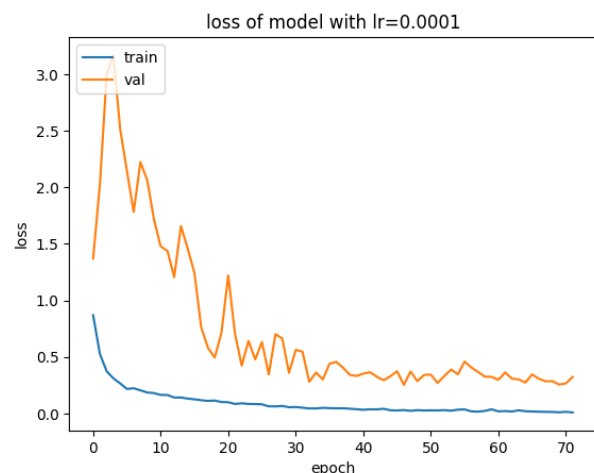
به نظر ما یکی از دلایل می‌تواند تفاوت در داده‌ها باشد. در داده مقاله کمتر از ۱۵۰ تصویر وجود دارد که پس از augmentation به ۹۰۰ تصویر میرسد ولی تعداد تصاویر ما بدون augmentation ۸۹۷ عدد است پس حتی شاید بتوان با ایجاد تغییراتی در مدل و انتخاب هایپرپارامترهای مناسب به دقت مقاله یا بهتر بدون augmentation دست یافت. همچنین در مقاله داده‌ها کم کیفیت توصیف شده‌اند و ممکن است کیفیت داده‌هایی که ما استفاده می‌کنیم بیشتر باشد و این یکی از دلایل عملکرد مدل باشد.

۱-۵-۲. آموزش با نرخ یادگیری ثابت

حال مدل را با سه نرخ یادگیری ثابت ولی با مقادیر مختلف روی تصاویر آموزش می‌دهیم. ابتدا نرخ یادگیری را برابر ۰.۰۰۰۱ قرار می‌دهیم.

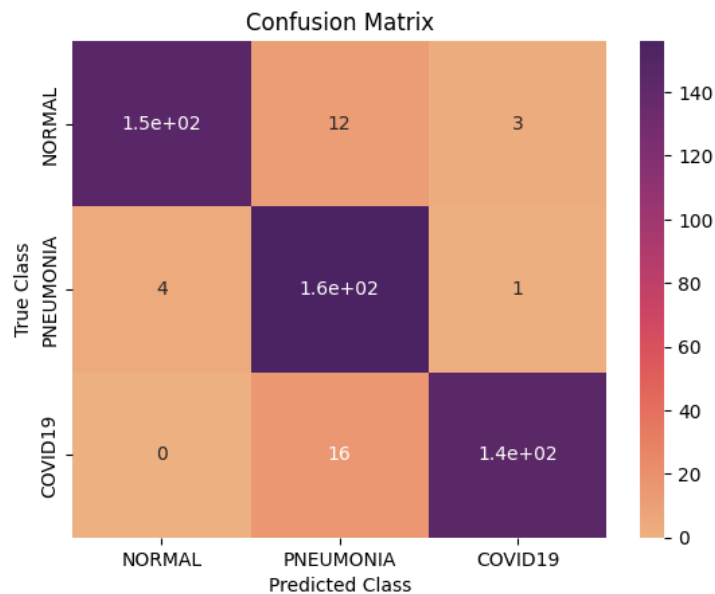


شکل ۹. صحت داده افزونه شده با نرخ یادگیری ۰.۰۰۰۱



شکل ۸. خطای داده افزونه شده با نرخ یادگیری ۰.۰۰۰۱

آموزش پس از ۷۲ اپیاک متوقف می‌شود. می‌توان دید با داده‌های افزونه، تغییرات خطا و صحت بسیار نرم‌تر و با نویز کمتر رخ داده است. دلیل آن می‌تواند به خاطر تعداد بیشتر داده‌ها یا استفاده از نرخ یادگیری کمتر باشد.

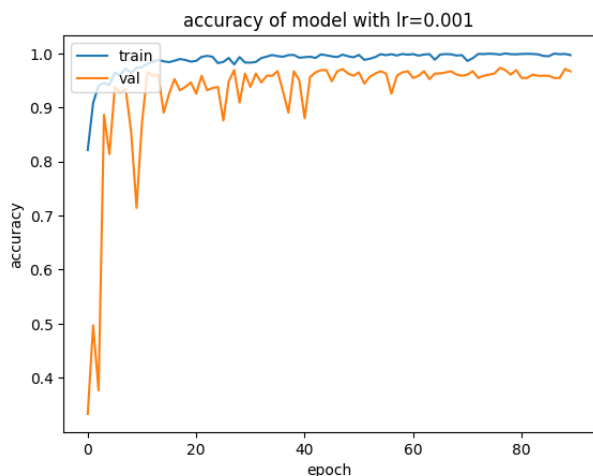


شکل ۱۰. ماتریس درهم‌ریختگی روی داده افزونه شده با نرخ یادگیری ۰.۰۰۰۱

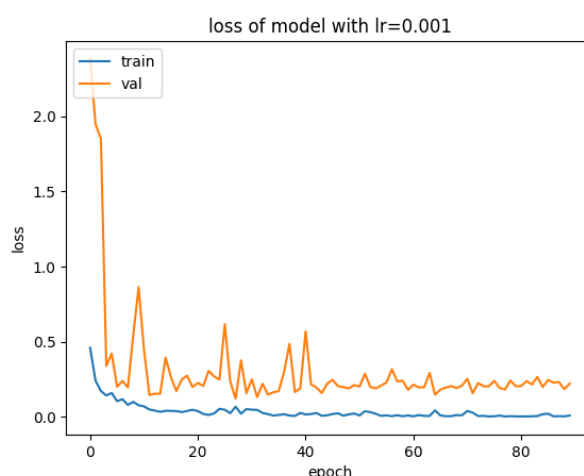
جدول ۴. معیارهای ارزیابی روی داده افزونه شده با نرخ یادگیری ۰.۰۰۰۱

	NORMAL	PNEUMONIA	COVID19	Micro	Macro
Accuracy	۹۰.۶۸	۹۶.۸۹	۹۰.۰۶	۹۲.۵۵	۹۲.۵۵
Precision	۹۷.۳۳	۸۴.۷۸	۹۷.۳۲	۹۲.۵۵	۹۳.۱۴
Recall	۹۰.۶۸	۹۶.۸۹	۹۰.۰۶	۹۲.۵۵	۹۲.۵۵
F1-score	۹۳.۸۹	۹۰.۴۳	۹۳.۵۵	۹۲.۵۵	۹۲.۶۲

می‌توان دید معیارهای ارزیابی حدود چهار درصد با افزودن داده کاهش یافته‌اند. البته یکی از دلایل عملکرد ضعیف‌تر می‌تواند توقف زودتر باشد چرا که مدل قبلی ۱۰۰ اپیاک آموزش دیده بود. ما چندین بار مدل‌ها را آموزش دادیم و به طور کلی به نظر می‌رسد تغییر زیادی در عملکرد مدل ایجاد نمی‌شود و بدون داده‌های افزونه عملکردی مشابه و یا قوی‌تر داشتیم. حال مدلی را با نرخ یادگیری ۰.۰۰۱ آموزش می‌دهیم.

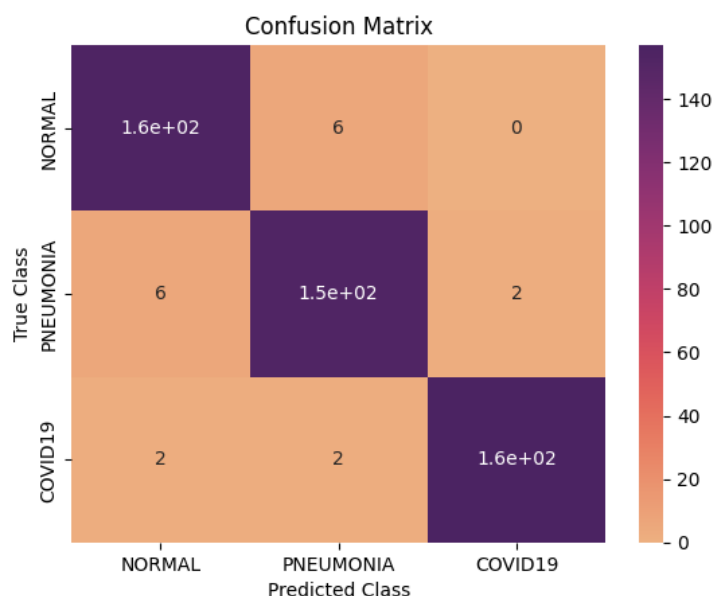


شکل ۱۲. صحت داده افزونه شده با نرخ یادگیری ۰.۰۰۱



شکل ۱۱. خطای داده افزونه شده با نرخ یادگیری ۰.۰۰۱

آموزش پس از ۹۰ اپیاک متوقف می‌شود.

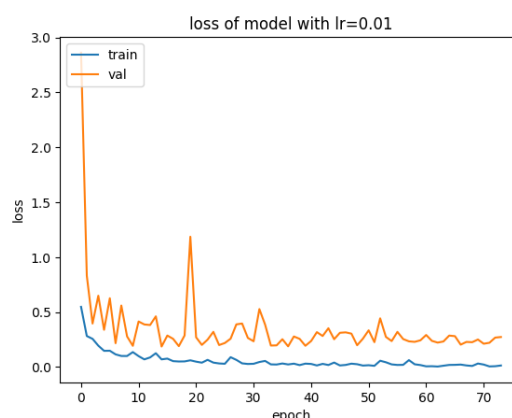
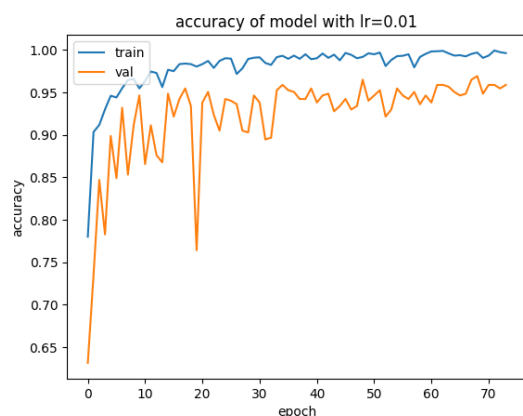


شکل ۱۳. ماتریس درهم‌ریختگی روی داده افزونه شده با نرخ یادگیری ۰.۰۰۱

جدول ۵. معیارهای ارزیابی روی داده افزونه شده با نرخ یادگیری ۰.۰۰۱

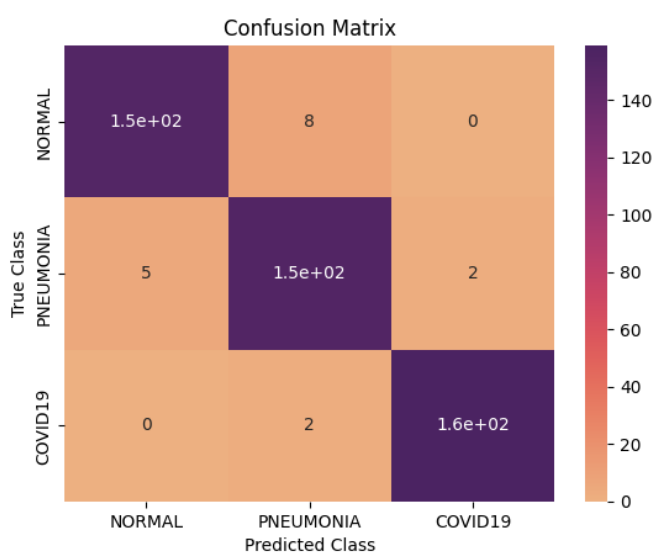
	NORMAL	PNEUMONIA	COVID19	Micro	Macro
Accuracy	۹۶.۲۷	۹۵.۰۳	۹۷.۵۲	۹۶.۲۷	۹۶.۲۷
Precision	۹۵.۰۹	۹۵.۰۳	۹۸.۷۴	۹۶.۲۷	۹۶.۲۹
Recall	۹۶.۲۷	۹۵.۰۳	۹۷.۵۲	۹۶.۲۷	۹۶.۲۷
F1-score	۹۵.۶۸	۹۵.۰۳	۹۸.۱۲	۹۶.۲۷	۹۶.۲۸

می‌توان دید نسبت به نرخ یادگیری ۰.۰۰۰۱ کمی عملکرد مدل بهتر شده که ممکن است به دلیل کند بودن آموزش مدل قبل یا گیر افتادن در نقاط بهینه محلی باشد ولی با این حال عملکرد مدل آن قدری اختلاف ندارد. حال نرخ یادگیری را برابر ۰.۰۱ قرار می‌دهیم و مدلی را آموزش می‌دهیم.



شکل ۱۵. صحت داده افزونه شده با نرخ یادگیری ۰.۰۱

شکل ۱۴. خطای داده افزونه شده با نرخ یادگیری ۰.۰۱



شکل ۱۶. ماتریس درهم‌ریختگی روی داده افزونه شده با نرخ یادگیری ۰.۰۱

جدول ۶. معیارهای ارزیابی روی داده افزونه شده با نرخ یادگیری ۰.۰۱

	NORMAL	PNEUMONIA	COVID19	Micro	Macro
Accuracy	۹۵.۰۳	۹۵.۶۵	۹۸.۷۶	۹۶.۴۸	۹۶.۴۸
Precision	۹۶.۸۴	۹۳.۹۰	۹۸.۷۶	۹۶.۴۸	۹۶.۵۰
Recall	۹۵.۰۳	۹۵.۶۵	۹۸.۷۶	۹۶.۴۸	۹۶.۴۸
F1-score	۹۵.۹۲	۹۴.۷۷	۹۸.۷۶	۹۶.۴۸	۹۶.۴۸

با بررسی این چهار مدل می‌توان نتیجه‌هایی گرفت. ابتدا می‌توان دید که چون کلاس‌ها را متعادل کردیم، تمام معیارهای ارزیابی مقدار بسیار مشابهی دارند و می‌توان با اندازه گیری یکی از آن‌ها به عملکرد مدل پی برد. همچنین همان طور که دیدیم، با اضافه کردن داده‌های افزونه، مقدار معیارهای عملکرد تغییر زیادی نکرد و هنوز با عملکرد مدل مقاله فاصله داریم. البته باید در نظر داشت که کلاس بیماران ذات‌الریه در داده‌های مقاله وجود نداشت و بدون در نظر گرفتن خطاهای این کلاس، به عملکرد مشابهی می‌رسیم.

به طور کلی نرخ یادگیری نشان‌دهنده سرعت تغییر وزن‌های مدل است. اگر نرخ یادگیری عددی بزرگ انتخاب شود، وزن‌ها با شدت بیشتری تغییر می‌کنند و با انتخاب مقداری بیش از حد بزرگ ممکن است مدل ناپایدار شود. از طرف دیگر اگر نرخ یادگیری خیلی کوچک باشد، یادگیری بسیار کند می‌شود و ممکن است مدل در نقاط بهینه محلی گیر بیفتد و نتواند خارج شود و نقاط بهتری را پیدا کند. سه نرخ یادگیری رایج ۰.۰۱، ۰.۰۰۱ و ۰.۰۰۰۱ هستند که ما هر سه را آزمودیم. با افزایش مقدار نرخ یادگیری عملکرد مدل‌ها افزایش یافت که این می‌تواند به دلیل افزایش سرعت آموزش باشد و احتمال دارد اگر مدل‌ها با نرخ یادگیری کوچک‌تر را بیشتر آموزش دهیم به عملکردی مشابه برسند البته اگر در نقاط بهینه محلی گیر نیفتاده باشند.

برای اینکه بتوان از مزیت هر دو نرخ یادگیری با مقدار کم و زیاد استفاده کرد روش‌های مختلفی ایجاد شده‌اند که نرخ یادگیری را متغیر در نظر می‌گیرند و طبق تابعی خاص مقدار آن را آپدیت می‌کنند که در ادامه بررسی خواهیم کرد.

۱-۵-۳. آموزش با نرخ یادگیری متغیر

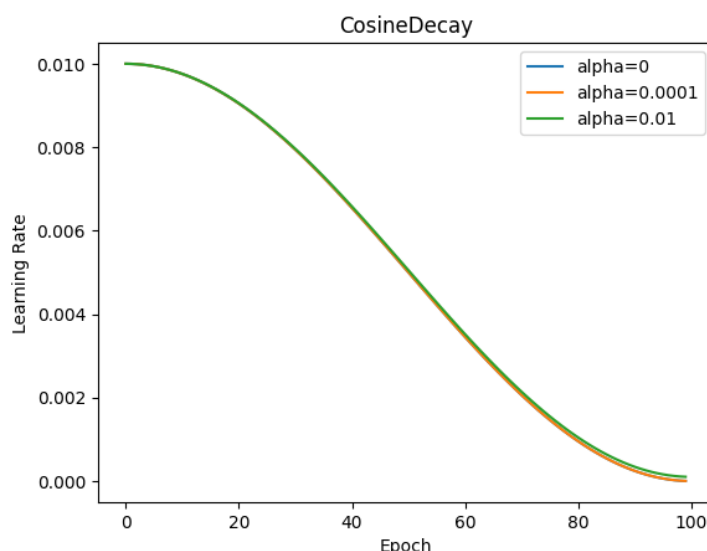
به طور کلی در اکثر روش‌هایی که نرخ یادگیری متغیر است، ابتدا نرخ یادگیری مقدار بزرگی در نظر گرفته می‌شود تا بتوان نقاط بهینه را با سرعت بیشتری پیدا کرد بدون آنکه مدل در آن‌ها گیر بیفتد و با

گذر زمان نرخ یادگیری را کاهش می‌دهیم تا پس از قرار گرفتن نزدیک نقطه بهینه، بدون گذر از آن به آن نزدیک شویم.

پارامتر `initial_lr` بین بسیاری از نرخ یادگیری‌ها مشترک است که نشان‌دهنده مقدار اولیه نرخ یادگیری است که ما مقدار ۰.۰۱ را برای آن در نظر می‌گیریم. پارامتر `decay_steps` هم در برخی مشترک است که نشان‌دهنده تعداد ایپاک‌هایی است که نرخ یادگیری متغیر است. ما برای باقی نرخ‌های یادگیری ۵۰ ایپاک اول یعنی حداکثر نیمی از مدت آموزش را برای تغییر نرخ یادگیری در نظر گرفتیم ولی برای این تابع، عملکرد مناسبی نگرفتیم و `decay_steps` را ۱۰۰ ایپاک در نظر گرفتیم تا نرخ یادگیری با سرعت کمتری کاهش یابد.

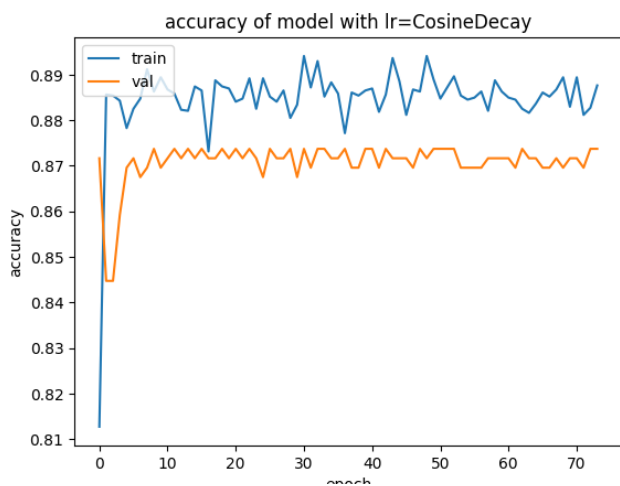
ابتدا نرخ یادگیری Cosine decay را بررسی می‌کنیم. رابطه کلی آن به این صورت است:

$$initial_lr \times \left(\frac{1}{2} (1 - \alpha) (1 + \cos(\frac{\pi Step}{DecaySteps})) + \alpha \right)$$



شکل ۱۷. تغییرات نرخ یادگیری طبق CosineDecay

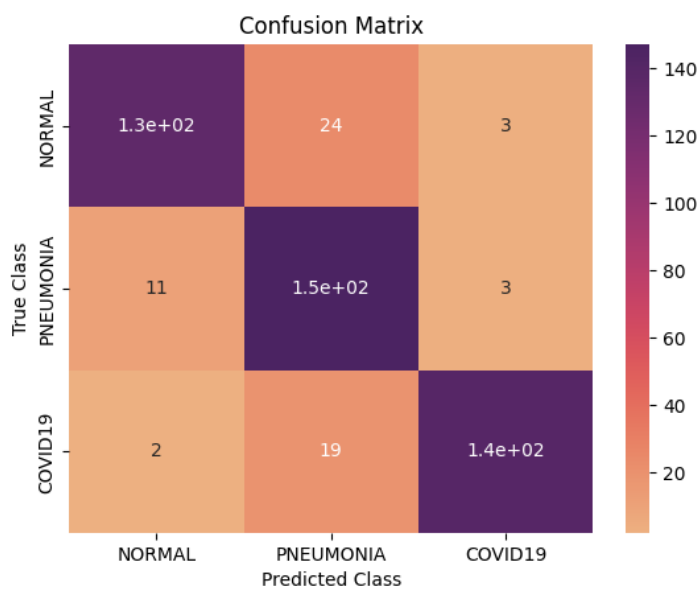
می‌توان دید که با ثابت بودن نرخ یادگیری اولیه و تعداد ایپاک‌های تغییر، پارامتر آلفا تغییر زیادی ایجاد نمی‌کند و ما آن را صفر در نظر می‌گیریم و مدلی را آموزش می‌دهیم.



شکل ۱۹. صحت مدل با نرخ یادگیری CosineDecay



شکل ۱۸. خطای مدل با نرخ یادگیری CosineDecay



شکل ۲۰. ماتریس درهم ریختگی مدل با نرخ یادگیری CosineDecay

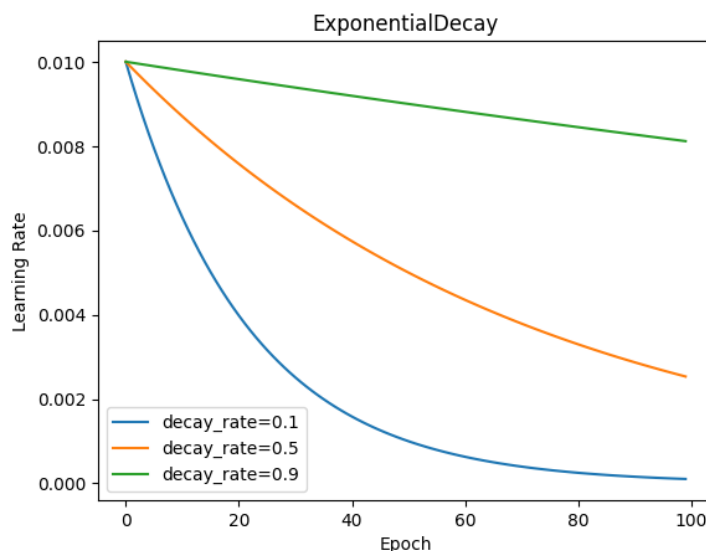
جدول ۷. معیارهای ارزیابی مدل با نرخ یادگیری CosineDecay

	NORMAL	PNEUMONIA	COVID19	Micro	Macro
Accuracy	۸۳.۲۳	۹۱.۳۰	۸۶.۹۶	۸۷.۱۶	۸۷.۱۶
Precision	۹۱.۱۶	۷۷.۳۷	۹۵.۸۹	۸۷.۱۶	۸۸.۱۴
Recall	۸۳.۲۳	۹۱.۳۰	۸۶.۹۶	۸۷.۱۶	۸۷.۱۶
F1-score	۸۷.۰۱	۸۳.۷۶	۹۱.۲۱	۸۷.۱۶	۸۷.۳۳

می‌توان دید مدل عملکردی مشابه با مدل‌های قبلی دارد. حال سراغ روش بعدی برای تعیین نرخ یادگیری می‌رویم.

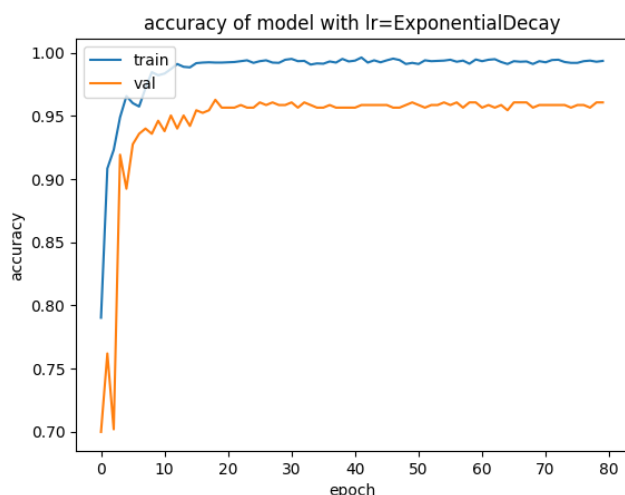
روش بعدی Exponential decay است که رابطه آن بدین صورت است:

$$initial_lr \times decay_rate^{\frac{Step}{DecaySteps}}$$

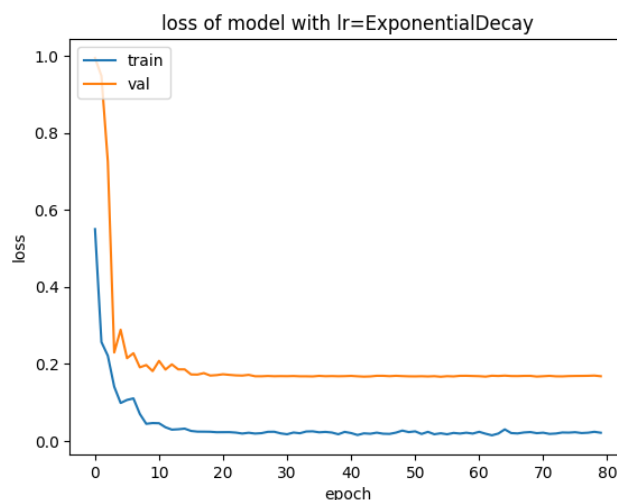


شکل ۲۱. تغییرات نرخ یادگیری طبق ExponentialDecay

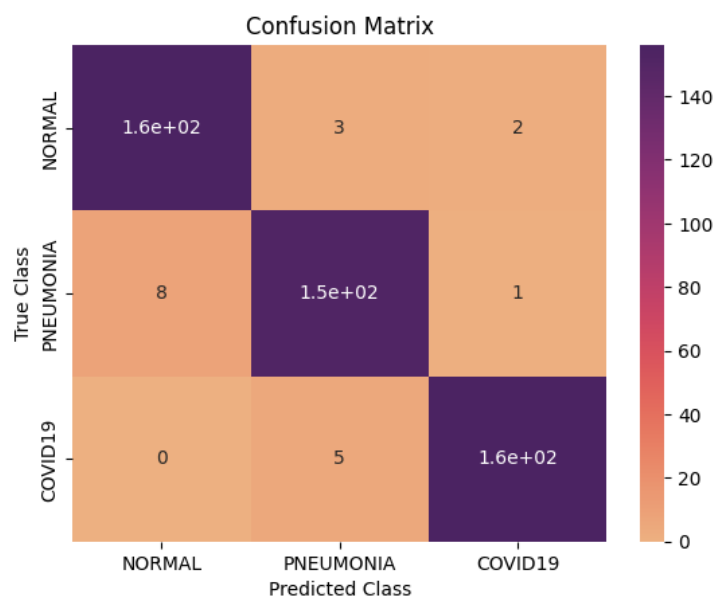
می‌توان دید با افزایش decay_rate، سرعت کاهش نرخ یادگیری کاهش می‌یابد. از آنجاییکه با کاهش decay_rate تغییرات به صورت خطی می‌شوند و خاصیت نمایی بودن را از دست می‌دهند و از طرفی معمولاً آموزش مدل بسیار زودتر از ۱۰۰ اپاک تمام می‌شود، ما decay_rate را برابر ۰.۹ در نظر می‌گیریم و مدل را آموزش می‌دهیم.



شکل ۲۳. صحت با نرخ یادگیری ExponentialDecay



شکل ۲۲. خطا با نرخ یادگیری ExponentialDecay



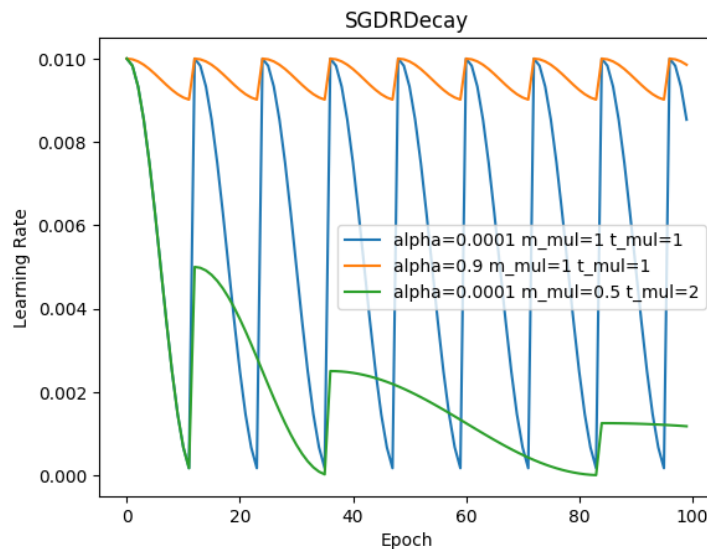
شکل ۲۴. ماتریس درهم‌ریختگی مدل با نرخ‌یادگیری ExponentialDecay

جدول ۸. معیارهای ارزیابی مدل با نرخ‌یادگیری ExponentialDecay

	NORMAL	PNEUMONIA	COVID19	Micro	Macro
Accuracy	۹۶.۸۹	۹۴.۴۱	۹۶.۸۹	۹۶.۰۷	۹۶.۰۷
Precision	۹۵.۱۲	۹۵.۰۰	۹۸.۱۱	۹۶.۰۷	۹۶.۰۸
Recall	۹۶.۸۹	۹۴.۴۱	۹۶.۸۹	۹۶.۰۷	۹۶.۰۷
F1-score	۹۶.۰۰	۹۴.۷۰	۹۷.۵۰	۹۶.۰۷	۹۶.۰۷

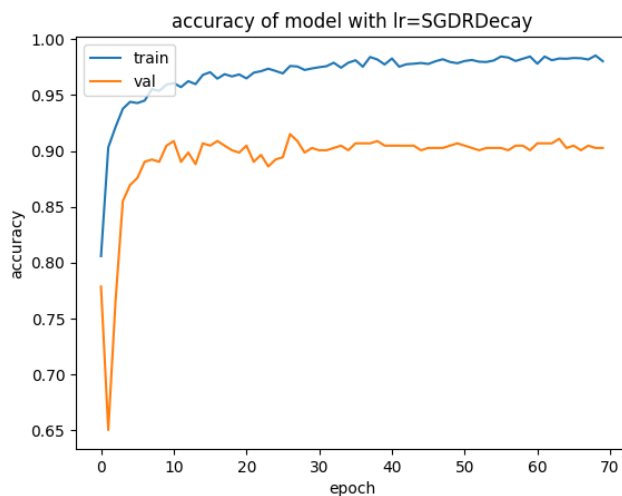
باز هم به مدلی با عملکردی مشابه می‌رسیم.

در برخی روش‌ها از توابع متناوب برای تعیین مقدار نرخ‌یادگیری استفاده می‌کنیم که باعث می‌شود به طور متناوب به مدل کمک شود که از نقاط بهینه محلی خارج شود. روش بعدی Cosine decay restarts است که رابطه آن مشابه Cosine decay است ولی به طور متناوب تکرار می‌شود.

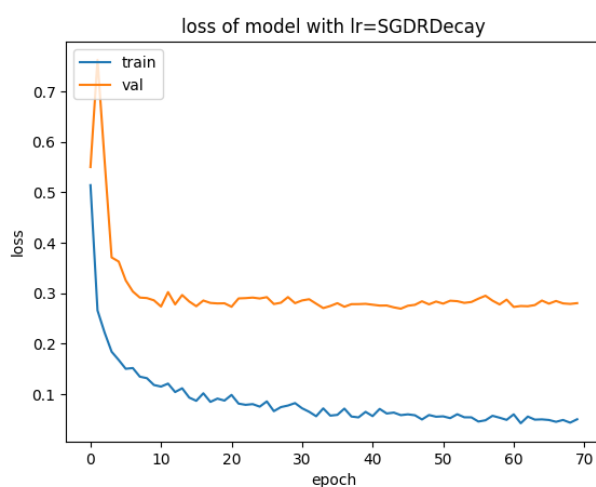


شکل ۲۵. تغییرات نرخ یادگیری طبق CosineDecayRestarts

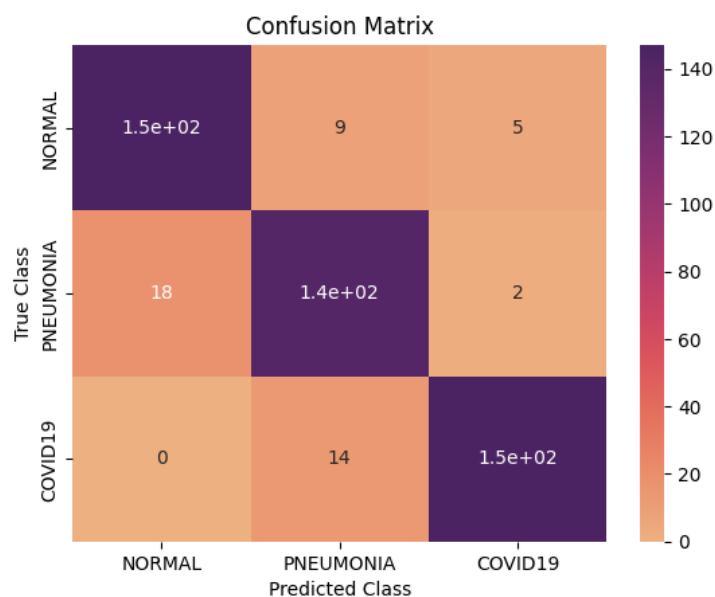
ما از تابع سبز رنگ در تصویر بالا استفاده می‌کنیم که با اینکه مقدار آن پس از رسیدن به پایین‌ترین حد خود دوباره زیاد می‌شود ولی به مقداری کمتر از ماکسیمم قبلی می‌رود و بدین صورت با گذر زمان نرخ یادگیری کاهش می‌یابد.



شکل ۲۷. صحت با نرخ یادگیری CosineDecayRestart



شکل ۲۶. خطا با نرخ یادگیری CosineDecayRestart



شکل ۲۸. ماتریس درهم‌ریختگی مدل با نرخ‌یادگیری CosineDecayRestarts

جدول ۹. معیارهای ارزیابی مدل با نرخ‌یادگیری CosineDecayRestarts

	NORMAL	PNEUMONIA	COVID19	Micro	Macro
Accuracy	۹۱.۳۰	۸۷.۵۸	۹۱.۳۰	۹۰.۰۶	۹۰.۰۶
Precision	۸۹.۰۹	۸۵.۹۸	۹۵.۴۵	۹۰.۰۶	۹۰.۱۷
Recall	۹۱.۳۰	۸۷.۵۸	۹۱.۳۰	۹۰.۰۶	۹۰.۰۶
F1-score	۹۰.۱۸	۸۶.۷۷	۹۳.۳۳	۹۰.۰۶	۹۰.۱۰

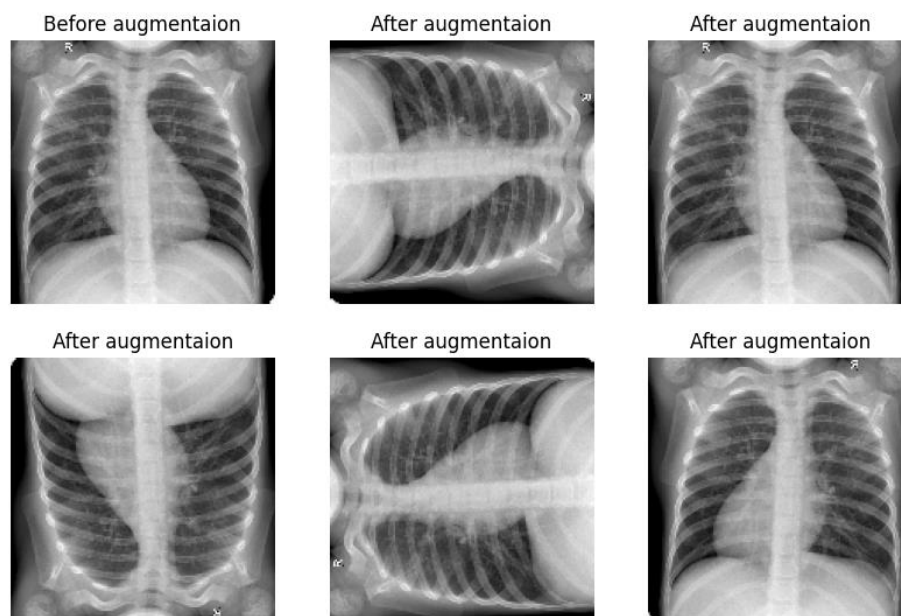
باز هم به عملکردی مشابه رسیدیم. به نظر می‌رسد برای این مدل و تصاویر و یا حتی خیلی از مدل‌ها و تصاویر دیگر، تفاوت زیادی بین روش‌های مختلف انتخاب نرخ‌یادگیری وجود نداشته باشد. با این حال احتمالاً انتخاب روش‌های تغییر نرخ‌یادگیری بتوانند هم باعث افزایش سرعت و هم افزایش صحت شوند ولی برای تصاویر مورد نظر ما، تمامی روش‌ها در مدت زمان نسبتاً کمی به صحت بالایی می‌رسند و تفاوت زیادی مشاهده نکردیم. البته به نظر می‌رسد نمودار تغییرات خطا و صحت با نرخ‌یادگیری متغیر نرم‌تر است و زودتر ثابت می‌شود و دلیل آن می‌تواند به این خاطر باشد که پس از گذر تعدادی اپیاک با نرخ‌یادگیری بزرگ، مدل نزدیک نقطه بهینه محلی قرار می‌گیرد و سپس با کم شدن نرخ‌یادگیری، تغییرات خطا و صحت کاهش می‌یابند.

با در نظر گرفتن ماتریس‌های درهم‌ریختگی رسم شده برای مدل‌ها می‌توان دید که به طور کلی بیشترین اشتباه مدل‌ها در تشخیص بیماران دارای ذات‌الریه است و شاید دلیل آن مشابهت تصویر ریه این دسته از افراد با افراد سالم و بیماران کرونایی باشد.

در مقاله کلاس بیماران ذات‌الریه وجود نداشت و اگر خطای مدل‌ها در تشخیص این کلاس را در نظر بگیریم، مدل‌هایی که آموزش دادیم به عملکرد مشابه مقاله رسیدند و حتی با وجود این کلاس هم عملکرد مدل‌ها بالای ۹۰ درصد است. عملکرد مدل‌ها حداکثر ۶ درصد تفاوت داشت و بهترین عملکرد را مدلی که با نرخ یادگیری ثابت با مقدار ۰.۰۱ آموزش دادیم نشان داد که عملکرد آن در جدول شماره ۶ نمایان است.

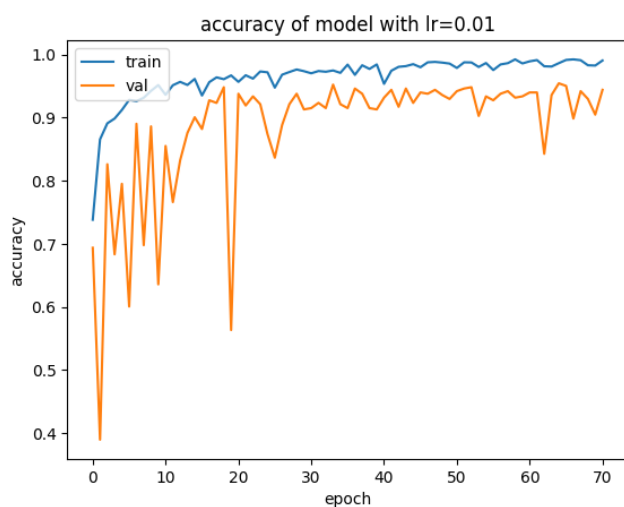
۱-۵-۴. ایجاد داده‌های افزونه با روش مقاله

همان طور که گفته شد عملکرد مدل‌های بالا بسیار نزدیک است ولی با توجه به معیار صحت، مدل با نرخ یادگیری ثابت ۰.۰۱ بهترین عملکرد را داشت و برای بررسی تفاوت در نحوه ایجاد داده‌های افزونه در روش ما و روش مقاله، یک بار هم داده‌ها را با روش مقاله ۵ برابر می‌کنیم و مدل را آموزش می‌دهیم. به نظر ما تاثیری در عملکرد نخواهد داشت چرا که همین الان هم می‌توانیم مشاهده کنیم مدل بدون داده‌های افزونه هم عملکردی مشابه دارد و افزونگی تاثیر زیادی ایجاد نکرده است که دلیل آن همان طور که بالاتر نوشتیم می‌تواند به خاطر تعداد بالاتر داده‌ها باشد.

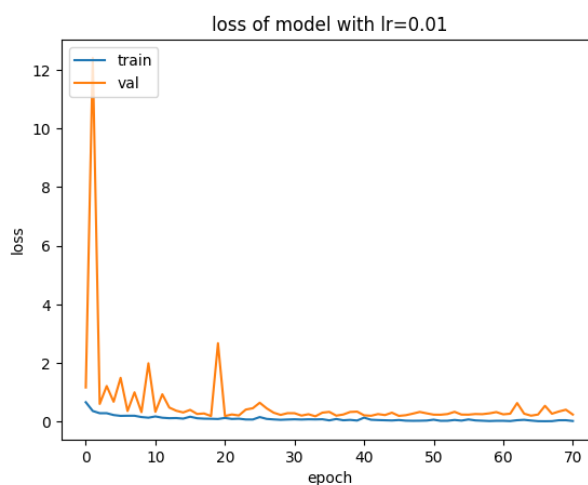


شکل ۲۹. نمونه‌ای از **data augmentation** مشابه مقاله

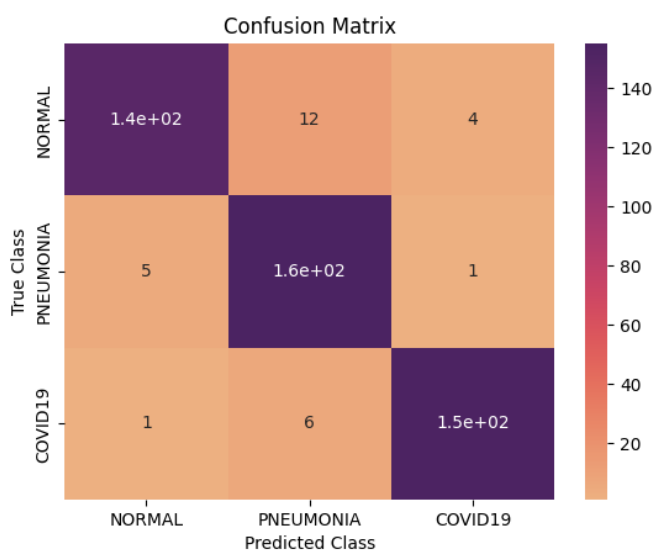
مدل را با داده‌های جدید اجرا می‌کنیم و پس از ۷۱ اپیاک آموزش متوقف می‌شود.



شکل ۳۱. تغییرات صحت با افزودن لرنیng به روش مقاله



شکل ۳۰. تغییرات خطا با افزودن لرنیng به روش مقاله



شکل ۳۲. ماتریس درهم‌ریختگی با افزودن لرنیng به روش مقاله

جدول ۱۰. معیارهای ارزیابی مدل با افزودن لرنیng به روش مقاله

	NORMAL	PNEUMONIA	COVID19	Micro	Macro
Accuracy	۹۰.۰۶	۹۶.۲۷	۹۵.۶۵	۹۴.۰۰	۹۴.۰۰
Precision	۹۶.۰۳	۸۹.۶۰	۹۶.۸۶	۹۴.۰۰	۹۴.۱۶
Recall	۹۰.۰۶	۹۶.۲۷	۹۵.۶۵	۹۴.۰۰	۹۴.۰۰
F1-score	۹۲.۹۵	۹۲.۸۱	۹۶.۲۵	۹۴.۰۰	۹۴.۰۰

همان طور که انتظار داشتیم عملکرد مدل فرق زیادی با مدل های قبلی ندارد.

۱-۵-۵. ارزیابی مدل های دیگر برای افزایش عملکرد

مدل های فعلی به مشکل بیش برآزش بر نمی خورند ولی با این حال در تشخیص کلاس ذات‌الریه مشکل دارند. به دلیل بالا بودن معیارها در حال حاضر و تعداد پارامترهای بالای مدل و رخ ندادن بیش برآزش، به نظر نمی رسد بتوان عملکرد را خیلی بهتر کرد. با این حال ما چندین تغییر اعمال کرده و مدل هایی را آموزش دادیم و ارزیابی کردیم.

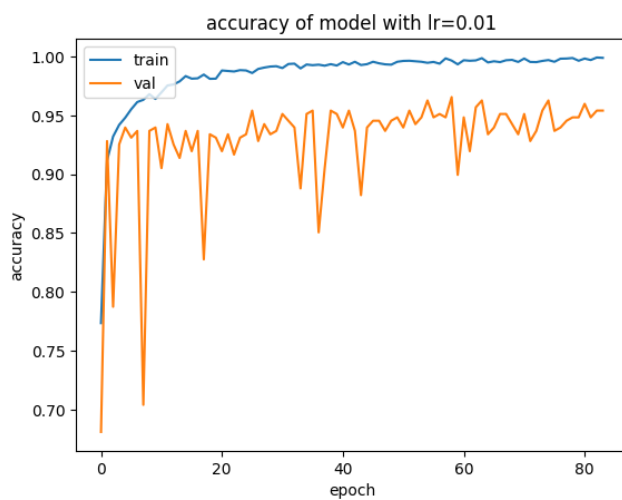
ابتدا نکته ای متوجه شدیم که به دلیل سیاه و سفید بودن تصاویر نیاز به سه کانال برای رنگ تصاویر نداریم و بدین صورت بدون حذف اطلاعات، ابعاد تصاویر را یک سوم کردیم. همچنین از همان روش افزودنی که ایجاد کردیم، استفاده می کنیم. همچنین به دلیل عملکرد بهتر مدل با نرخ یادگیری ۰.۰۱، این نرخ یادگیری را انتخاب می کنیم.

ابتدا مدلی را با تصاویر سیاه و سفید آموزش دادیم و عملکرد بهتری مشاهده نکردیم. سپس با افزودن لایه های بیشتر، تعداد لایه های feedforward انتهایی مدل را به شش رساندیم و باز هم عملکرد مدل افزایش پیدا نکرد. ایده بعدی استفاده از لایه های Conv2dTranspose بود که پس از لایه های کانولوشنی دو لایه deconvolution قرار دادیم ولی به طور مشابه به عملکردی مشابه رسیدیم. نهایتاً برای جلوگیری از حذف اطلاعات با تغییر ابعاد تصاویر، یک بار هم تصاویر را با ابعاد ۳۰۰ در ۳۰۰ لود و مدلی را آموزش دادیم ولی باز هم بهبودی حاصل نشد.

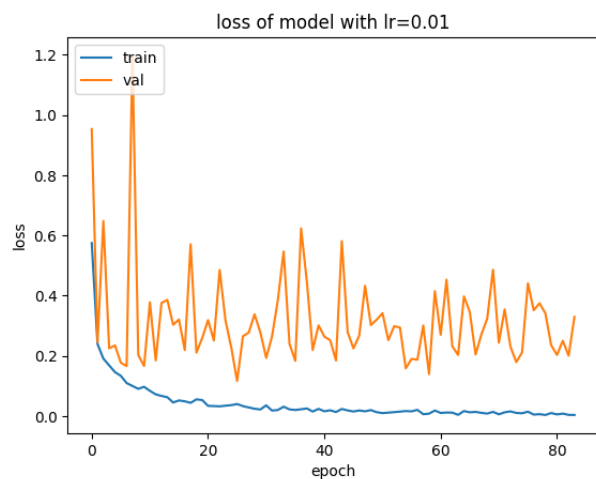
۱-۵-۶. ارزیابی نهایی

از آن جایی که عملکرد مدل ها بهتر نشد، حال بار دیگر مدل با نرخ یادگیری ثابت ۰.۰۱ را روی هر دو داده آموزش و validation آموزش می دهیم و روی داده تست، به عنوان عملکرد نهایی ارزیابی می کنیم. با این کار تعداد داده های آموزش برابر ۶۹۰۰ می شود و با مشکل حافظه GPU مواجه می شویم. به همین دلیل برای ارزیابی نهایی نیز از داده های با یک کانال یعنی تصاویر سیاه و سفید استفاده می کنیم.

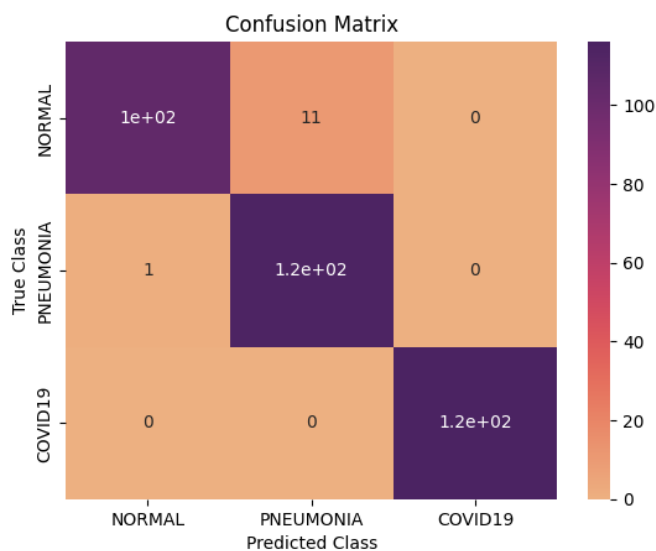
آموزش پس از ۸۴ اپاک متوقف می‌شود.



شکل ۳۴. تغییرات صحت ارزیابی روی داده تست



شکل ۳۳. تغییرات خطا روی داده تست



شکل ۳۵. ماتریس درهم‌ریختگی برای ارزیابی روی داده تست

جدول ۱۱. معیارهای ارزیابی مدل برای ارزیابی روی داده تست

	NORMAL	PNEUMONIA	COVID19	Micro	Macro
Accuracy	۹۰.۵۲	۹۹.۱۴	۱۰۰.۰۰	۹۶.۵۵	۹۶.۵۵
Precision	۹۹.۰۶	۹۱.۲۷	۱۰۰.۰۰	۹۶.۵۵	۹۶.۷۸
Recall	۹۰.۵۲	۹۹.۱۴	۱۰۰.۰۰	۹۶.۵۵	۹۶.۵۵
F1-score	۹۴.۵۹	۹۵.۰۴	۱۰۰.۰۰	۹۶.۵۵	۹۶.۵۵

در نهایت به مدلی با عملکرد بالا می‌رسیم. می‌توان دید که مدل هیچ‌گونه خطایی در تشخیص کلاس بیماران دارای کرونا ندارد. همچنین تنها از بین تصاویری که به عنوان سالم تشخیص داده شده‌اند، تنها یک مورد که مربوط به بیماران دارای ذات‌الریه است، به اشتباه سالم تشخیص داده شده است. اما بیشترین اشتباه در تشخیص افراد سالم بوده که ۱۱ مورد به اشتباه دارای ذات‌الریه تشخیص داده شده‌اند. می‌دانیم در واقعیت، برای تشخیص بیماری‌ها معیار بازیابی (recall) اهمیت بسیاری دارد و همان طور که مشخص است برای هر دو بیماری ذات‌الریه و کرونا، این معیار بالای ۹۹ درصد است که می‌تواند جان بسیاری را نجات دهد و کمتر از یک درصد امکان دارد که فرد بیمار را سالم تشخیص دهد. بدین ترتیب آموزش به موفقیت انجام شد و به عملکرد مورد نظر روی داده تست رسیدیم. حال بررسی می‌کنیم که آیا با یادگیری انتقالی هم می‌توانیم به همین عملکرد بالایی برسیم یا خیر.

۱-۶. یادگیری انتقالی

یادگیری انتقالی روشی در یادگیری ماشین و یادگیری عمیق است که در حوزه‌های مختلف به خصوص در بینایی ماشین و پردازش زبان‌های طبیعی کاربرد بسیاری دارد. در این روش مدلی که برای کار خاصی آموزش دیده را تغییر می‌دهیم تا روی کار مورد نظر ما نیز بتواند عملکرد خوبی داشته باشد.

یکی از مزیت‌های یادگیری انتقالی هزینه آموزش کمتر می‌باشد. ساخت مدل جدید علاوه بر سختی‌های انتخاب معماری مناسب، می‌تواند از لحاظ آموزش نیز هزینه‌بر باشد چرا که مدلی که هیچ داده‌ای ندیده است را می‌خواهیم آموزش دهیم درحالی‌که در یادگیری انتقالی مدل از قبل روی مجموعه داده‌ای آموزش دیده و بدین ترتیب می‌تواند در زمان کمتری روی مجموعه داده جدید آموزش ببیند. همچنین برخلاف یادگیری مدل جدید که نیاز به داده فراوان دارد، در یادگیری انتقالی چون مدل از قبل داده زیادی دیده، ویژگی‌ها مشترک تمام تصاویر مثل لبه‌ها را تشخیص می‌دهد پس با داده کمتر نیز می‌تواند به عملکرد بسیار خوبی برسد. به طور مشابه به دلیل آموزش روی داده فراوان از قبل، در یادگیری انتقالی از بیش‌برازش جلوگیری می‌شود چرا که مدل اطلاعاتی عمومی نسبت به واقعیت از طریق داده‌های مختلف کسب کرده است و قابلیت تعمیم آن افزایش می‌یابد.

البته برای رسیدن به عملکرد قابل قبول در یادگیری انتقالی توصیه می‌شود که کاری که قبل و بعد از آموزش روی مدل از آن می‌خواهیم انجام دهد تا حدی مشابه باشد و به طور مشابه توزیع داده ابتدایی و داده ما نیز تفاوت بسیار زیادی نداشته باشد.

به طور کلی سه نوع یادگیری انتقالی وجود دارد. در یادگیری انتقال استقرایی (Inductive transfer learning)، داده‌های مبدا و مقصد می‌تواند متفاوت یا مشابه باشند و هر دو برچسب‌گذاری شده‌اند و با

سعی می‌کنیم مشکل متفاوتی را حل کنیم مانند استفاده از مدل برچسب‌گذاری تصاویر برای هدف تشخیص اشیاء. در روش دوم که یادگیری انتقالی بدون نظارت (Unsupervised transfer learning) نام دارد، مشابه روش قبل است ولی در این حالت هر دو داده‌های مبدا و مقصد بدون برچسب می‌باشند. برای مثال می‌توان مدلی که الگوهای موجود در تصاویر را تشخیص می‌دهد انتخاب کرد و از آن برای تشخیص الگوهای خاص‌تر استفاده کرد. روش آخر یادگیری ترانسسانی (Transductive transfer learning) است که داده‌های مبدا برچسب‌گذاری شده‌اند در حالی که داده‌های مقصد بدون برچسبند. برای مثال می‌توان از مدلی که روی عقیده‌کاوی (sentiment analysis) کالاها آموزش دیده است برای عقیده‌کاوی نظرات کاربران در مورد فیلم‌ها استفاده کرد.

در بینایی ماشین معمولاً شبکه‌های کانولوشنی ابتدا ویژگی‌های پایه‌ای مانند لبه‌ها را در لایه‌های ابتدایی تشخیص می‌دهند. سپس در لایه‌های بعدی اشکال را تشخیص می‌دهند و در لایه‌های آخر ویژگی‌های منحصر به داده‌های آموزش را تشخیص می‌دهند. از آن جاییکه هر دو مدل VGG16 و MobileNetV2 روی داده‌های Imagenet که شامل تصاویر عمومی از کلاس‌های مختلف است آموزش دیده‌اند، پس ما لایه‌های fully connected انتهایی را حذف می‌کنیم و به جای آن‌ها لایه‌های جدیدی را قرار داده و آموزش می‌دهیم. ما یک لایه انتهایی با سه نورون برای تشخیص سه کلاس تصاویر در هر دو مدل قرار دادیم. یک لایه دیگر هم پیش از آن قرار دادیم و تعداد نورون‌های آن را جوری انتخاب کردیم که تعداد پارامترهای قابل آموزش در هر دو مدل بیشتر از مدل ابتدایی که ۱.۸ میلیون پارامتر بود، نشود.

۱-۶-۱. تصاویر ImageNet

این داده‌ها شامل بیش از ۱۴ میلیون تصویر است که توسط انسان‌ها برچسب‌گذاری شده‌اند و در حداقل یک میلیون از آن‌ها جعبه فراگیر شیء مورد نظر نیز وجود دارد. تصاویر از بیش از ۲۰ هزار کلاس مختلف تشکیل شده‌اند و از کلاس‌های معمول مثل توت فرنگی و بادکنک چندصد تصویر وجود دارد. این مجموعه داده از تصاویر موجود در اینترنت با ابعاد متنوع تشکیل شده است. همچنین بعضی از برچسب‌ها ویژگی‌های موجود در تصویر را شامل می‌شوند.

از سال ۲۰۱۰ سالانه مسابقه‌ای برای تشخیص این تصاویر برگزار می‌شود که در سال ۲۰۱۲ مدل معروف AlexNet با استفاده از شبکه‌های کانولوشنی توانست حدود ۱۰ درصد خطا را کاهش دهد و باعث افزایش توجه به سمت این مدل‌ها و افزایش استفاده از آن‌ها در بینایی ماشین شود.

۱-۶-۲. VGG16

در مسابقه سال ۲۰۱۴ تصاویر Imagenet، مدل VGG16 معرفی شد که در تشخیص اشیاء اول و در برچسب گذاری دوم شد. از آن پس این مدل به عنوان کانولوشنی یکی از بهترین مدل های بینایی ماشین شناخته می شود و در یادگیری انتقالی از آن بسیار استفاده می شود. سازندگان مدل با کاهش سایز فیلتر به ۳ در ۳ توانستند تعداد لایه ها را به ۱۶ لایه افزایش دهند و به عملکرد خوبی برسند. عدد ۱۶ در نام مدل هم به همین ۱۶ لایه اشاره دارد. مدل از ۱۳ لایه کانولوشنی، ۵ لایه max pooling و ۳ لایه عمیق تشکیل شده است که در کل ۲۱ لایه می شود ولی تنها ۱۶ لایه قابل آموزش هستند. این مدل تصاویر را به ابعاد ۲۲۴ در ۲۲۴ با ۳ کانال به صورت RGB ورودی می گیرد.

تبدیل تصاویر به ابعاد ۲۲۴ در ۲۲۴ به سه کانال و افزونه کردن تصاویر، باعث اتمام حافظه GPU می شود پس ما از مرحله افزودنی تصاویر صرف نظر کردیم. بدین ترتیب ۱۳۸۰ داده آموزش و ۳۴۸ داده تست با ابعاد ۲۲۴ در ۲۲۴ پیکسل داریم. حال مدل VGG16 را بدون قسمت fully connected انتهایی لود می کنیم و قسمت کانولوشنی آن را فریز می کنیم و مدل با معماری زیر را تشکیل می دهیم.

جدول ۱۲. خلاصه معماری مدل تشکیل شده از VGG16 برای یادگیری انتقالی

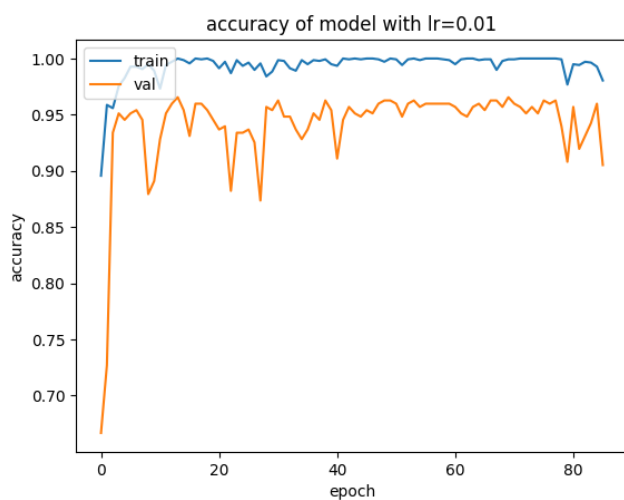
Layer (type)	Output Shape	Param #
input_layer_13 (InputLayer)	(None, 224, 224, 3)	0
VGG16	(None, 7, 7, 512)	14,714,816
flatten_10 (Flatten)	(None, 25088)	0
dense_29 (Dense)	(None, 64)	1,605,696
batch_normalization_19 (BatchNormalization)	(None, 64)	256
re_lu_19 (ReLU)	(None, 64)	0
dense_30 (Dense)	(None, 3)	195

Total params: 16,320,835 (62.26 MB)

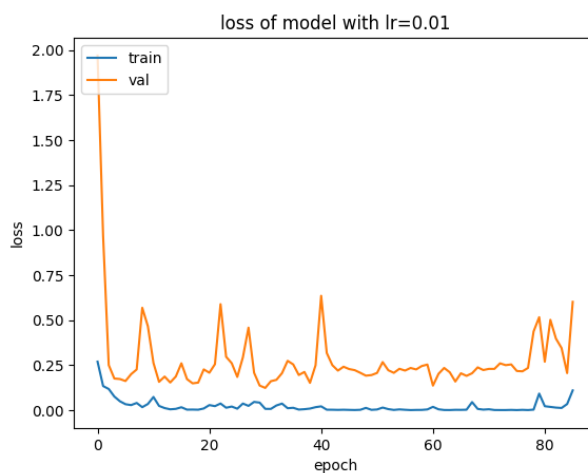
Trainable params: 1,606,019 (6.13 MB)

Non-trainable params: 14,714,816 (56.13 MB)

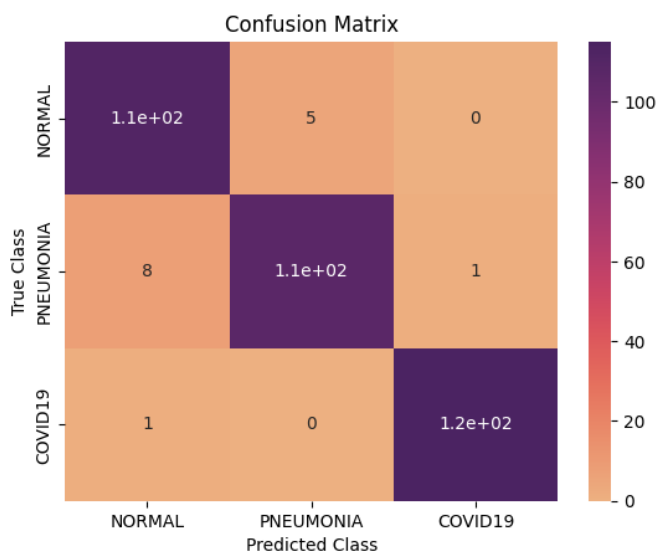
می توان دید که تعداد پارامترهای قابل آموزش کمتر از تعداد پارامترهای قابل آموزش در مدل های قبلی است. حال این مدل را هم آموزش می دهیم.



شکل ۳۷. تغییرات صحت VGG16 روی داده تست



شکل ۳۶. تغییرات خطای VGG16 روی داده تست



شکل ۳۸. ماتریس درهم‌ریختگی VGG16 برای ارزیابی روی داده تست

جدول ۱۳. معیارهای ارزیابی مدل VGG16 برای ارزیابی روی داده تست

	NORMAL	PNEUMONIA	COVID19	Micro	Macro
Accuracy	۹۵.۶۹	۹۲.۲۴	۹۹.۱۴	۹۵.۶۹	۹۵.۶۹
Precision	۹۲.۵۰	۹۵.۵۴	۹۹.۱۴	۹۵.۶۹	۹۵.۷۲
Recall	۹۵.۶۹	۹۲.۲۴	۹۹.۱۴	۹۵.۶۹	۹۵.۶۹
F1-score	۹۴.۰۷	۹۳.۸۶	۹۹.۱۴	۹۵.۶۹	۹۵.۶۹

آموزش مدل پس از ۸۶ اپاک متوقف می‌شود و می‌توان دید به طور مشابه به مدلی با عملکرد قابل قبول می‌رسیم. نکته قابل توجه این است که صحت مدل روی داده تست پس از اپاک سوم بیشتر از ۹۳ درصد می‌شود که این نشان دهنده سرعت بالای آموزش به دلیل آموزش دیدن از قبل و داشتن لایه‌هایی برای استخراج ویژگی‌ها است. بدین ترتیب انتقال یادگیری از تصاویر Imagenet به تصاویر سه کلاسه خودمان موفقیت آمیز بود و به مدلی با عملکرد مناسب رسیدیم.

۱-۶-۳. MobileNetV2

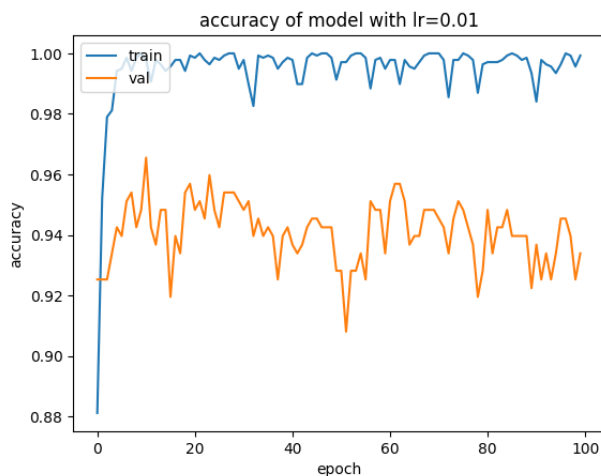
این مدل توسط پژوهشگران گوگل با هدف کار بر موبایل‌ها و سیستم‌های نهفته ایجاد شد. به همین دلیل از روش‌های مختلفی برای کاهش اندازه و افزایش سرعت و عملکرد مدل استفاده شده است. بدین ترتیب قسمت fully connected انتهایی مدل را حذف و قسمت ابتدایی را فریز می‌کنیم و قسمت fully connected جدید را اضافه می‌کنیم و به مدل زیر می‌رسیم.

جدول ۱۴. خلاصه معماری مدل تشکیل شده از MobileNetV2 برای یادگیری انتقالی

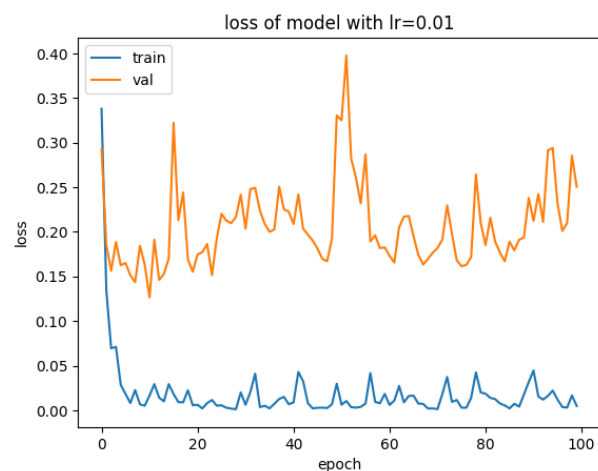
Layer (type)	Output Shape	Param #	Connected to
input_layer_14 (InputLayer)	(None, 224, 224, 3)	0	-
MobileNetV2	(None, 7, 7, 1280)	2,258,016	input_layer_14[0][0]
flatten_17 (Flatten)	(None, 62720)	0	out_relu[0][0]
dense_43 (Dense)	(None, 16)	1,003,536	flatten_17[0][0]
batch_normalization_26 (BatchNormalization)	(None, 16)	64	dense_43[0][0]
re_lu_26 (ReLU)	(None, 16)	0	batch_normalization
dense_44 (Dense)	(None, 3)	51	re_lu_26[0][0]

Total params: 3,261,635 (12.44 MB)
 Trainable params: 1,003,619 (3.83 MB)
 Non-trainable params: 2,258,016 (8.61 MB)

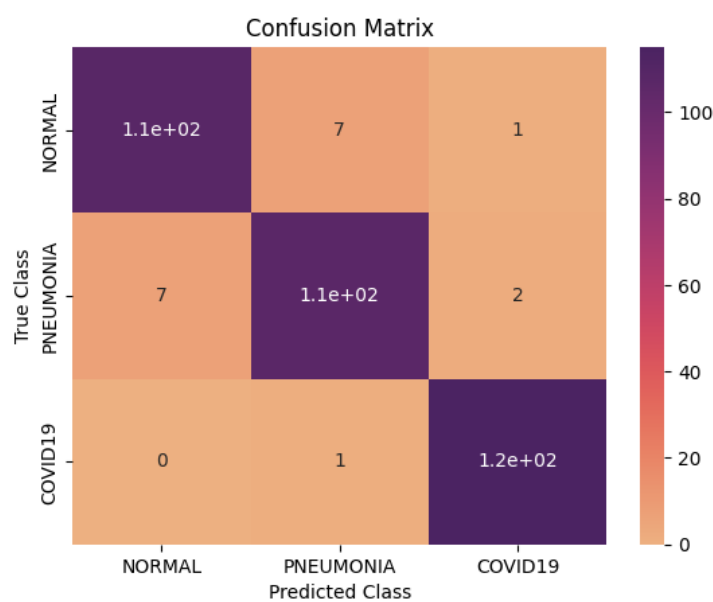
مدل MobileNetV2 تعداد لایه‌های بسیار بیشتری از مدل VGG16 دارد ولی به دلیل تکنیک‌های استفاده شده و نوع لایه‌ها تعداد پارامترهای آن کمتر از یک ششم مدل VGG16 است. ما تعدادی لایه fully connected به انتهای مدل به نحوی اضافه می‌کنیم که تعداد پارامترهای آموزش از مدل اولی که استفاده کردیم بیشتر نشود. واز تصاویر استفاده شده برای آموزش VGG16 هم برای آموزش این مدل استفاده می‌کنیم.



شکل ۳۹. تغییرات صحت MobileNetV2



شکل ۴۰. تغییرات خطای MobileNetV2



شکل ۴۱. ماتریس درهم‌ریختگی MobileNetV2 برای ارزیابی روی داده تست

جدول ۱۵. معیارهای ارزیابی مدل MobileNetV2 برای ارزیابی روی داده تست

	NORMAL	PNEUMONIA	COVID19	Micro	Macro
Accuracy	۹۳.۱۰	۹۲.۲۴	۹۹.۱۴	۹۴.۸۳	۹۴.۸۳
Precision	۹۳.۹۱	۹۳.۰۴	۹۷.۴۶	۹۴.۸۳	۹۴.۸۰
Recall	۹۳.۱۰	۹۲.۲۴	۹۹.۱۴	۹۴.۸۳	۹۴.۸۳
F1-score	۹۳.۵۱	۹۲.۶۴	۹۸.۲۹	۹۴.۸۳	۹۴.۸۱

باز هم می‌توان دید به عملکرد مناسبی می‌رسیم و در همان ایپاک اول صحت داده تست بالای ۹۲ درصد می‌شود که می‌تواند به دلیل از قبل آموزش دیدن مدل روی تصاویر دیگر باشد.

۷-۱. جمع‌بندی

در این تمرین تاثیر معماری‌های مختلف و افزونگی‌های مختلف برای آموزش تصاویر پزشکی با سه کلاس افراد سالم، بیماران دارای ذات‌الریه و بیماران دارای کووید ۱۹ را بررسی کردیم. همچنین تعداد زیادی از نرخ‌های یادگیری متفاوت را بررسی کردیم. تمامی مدل‌ها عملکرد قابل قبول بالای ۹۰ درصد داشتند و در بهترین حالت عملکردشان به بالای ۹۶ درصد هم رسید. نهایتاً با استفاده از مدل‌های VGG16 و MobileNetV2 نشان دادیم حتی با پارامترهای آموزش کمتر نیز به کمک یادگیری انتقالی می‌توانیم به مدل‌های با کیفیتی برسیم.

حال سه مدلی که روی داده تست بررسی کردیم یعنی مدل ساخته شده در مقاله، مدل VGG16 و مدل MobileNetV2 با روش یادگیری انتقالی را بررسی می‌کنیم. هر سه مدل با نرخ یادگیری ثابت ۰.۰۱ آموزش داده شدند. به دلیل محدودیت‌های GPU برای مدل‌هایی که با یادگیری انتقالی آموزش دادیم را از داده‌های افزونه استفاده نکردیم. با توجه به متعادل بودن کلاس‌ها تمام معیارهای ارزیابی مقدار مشابهی دارند پس ما صحت را به عنوان معیار اصلی در نظر می‌گیریم.

جدول ۱۶. مقایسه سه مدل با ارزیابی روی داده تست

	Accuracy	First epoch to reach 90% accuracy on test data	Total params	Trainable params
Model used in paper	۹۶.۵۵	۲	۱,۸۰۸,۹۶۳	۱,۸۰۵,۶۳۵
VGG16	۹۵.۶۹	۳	۱۶,۳۲۰,۰۱۹	۱,۶۰۶,۰۱۹
MobileNetV2	۹۴.۸۳	۱	۳,۲۶۱,۶۳۵	۱,۰۰۳,۶۱۹

می‌توان دید هر سه مدل به سرعت توانستند به صحت بالای ۹۰ درصد برسند و در نهایت هم عملکرد مشابهی دارند ولی مدل استفاده شده در مقاله کمی عملکرد بهتری دارد. عملکرد بهتر مدل می‌تواند به دلیل تعداد پارامترهای آموزشی بیشتر و آموزش لایه‌های کانولوشنی، استفاده از داده‌های افزونه و در نتیجه دیدن ۵ برابر داده نسبت به دو مدل دیگر و یا به دلیل تفاوت معماری استفاده شده باشد. می‌توان دید تعداد کل پارامترهای مدل VGG16 بسیار بیشتر از دو مدل دیگر است ولی تعداد پارامترهای قابل آموزش دو مدل VGG16 و MobileNetV2 همان طور که خواستیم کمتر است. بدین ترتیب بهترین عملکرد را می‌توان مدل استفاده شده در مقاله با نرخ یادگیری ثابت ۰.۰۱ در نظر گرفت.